Memoria de desarrollo de la prueba de concepto (PoC) Módulo de reportes

Objeto del proyecto:

Proyecto para crear una prueba de concepto de uso de consultas de base de datos basada en filtros y su posterior exportación a un csv para abrir con Excel usando Laravel como Backend y Vue como Frontend.

Contexto de Negocio:

Este módulo de reportes es un SaaS simulado para la gestión de incidencias municipales. Los operarios registran averías (fugas de agua, alumbrado, limpieza) y los supervisores necesitan analizar estos datos mensualmente.

El problema técnico que quiero simular es que el histórico de incidencias supera los 50.000 registros anuales. La herramienta actual se bloquea (TimeOut) al intentar descargar los reportes mensuales en Excel, y los caracteres especiales (tildes, ñ) se rompen al abrirse en versiones antiguas de Excel.

Esta PoC de un SaaS con un módulo que permite filtrar incidencias por categoría y rango de fechas, generando archivos CSV compatibles con Excel (UTF-8 BOM) mediante streamings de datos para no saturar la memoria RAM del servidor, todo bajo una arquitectura SPA (Single Page Application).

Patrón de arquitectura

Para el patrón de arquitectura evalué implementar una Arquitectura Hexagonal (Puertos y Adaptadores) para mantener el dominio agnóstico. Sin embargo, viendo cómo trabaja Laravel y su patrón Active Record, donde el modelo sabe cómo guardarse en la base de datos, si quisiera crear las Entidades puras, debería escribir Repositorios, Interfaces, DTOs y Mapeadores solo para pasar datos de Laravel a mi dominio, es decir, que para una prueba de concepto tendría que escribir el triple de código para hacer lo mismo.

Sin embargo, sí que he optado por una arquitectura "híbrida" una MVC respaldada por un Patrón de Servicios con lo que aprovecharía la velocidad de desarrollo del patrón Active Record nativo de Laravel, pero encapsulo la lógica de negocio en servicios aislados. Así garantizo al menos el principio de Responsabilidad única de SOLID.

Diseño de las Entidades y Modelo Entidad-Relación (ER)

Las 3 Entidades Principales:

1- Usuario(User):

Va ha tener un rol técnico o administrativo que va a gestionar las incidencias.

Sería necesario saber "quién" se encargará de cada incidencia.

Los atributos que serían clave:

- ID
- Nombre
- Email

2- Las categorías de la incidencia (Category):

Tipificarán el problema ("Saneamiento", "Alumbrado", "Jardinería").

Es un campo que me servirán para aplicar los filtros.

Los atributos clave serían:

- ID
- Nombre
- Prioridad (Alta, Media, Baja).

3- Incidencia (Ticket):

La tabla de “hechos”.

Conectará el operario (el quién) con la categoría (el qué) y con el Timestamps determinaré en qué momento se crea la incidencia (el cuándo).

Los atributos clave serían:

- ID
- Título
- Descripción
- Estado (Abierto/cerrado)
- Nombre del ciudadano
- Fecha de creación
- Fecha de eliminación
- Fecha de completado

Entidad Etiqueta (Tag)

Para un filtrado más eficiente se pueden añadir etiquetas a los tickets

Los atributos clave para las etiquetas serían:

- Id
- Nombre
- Color

Entidad Adjunto (Attachment)

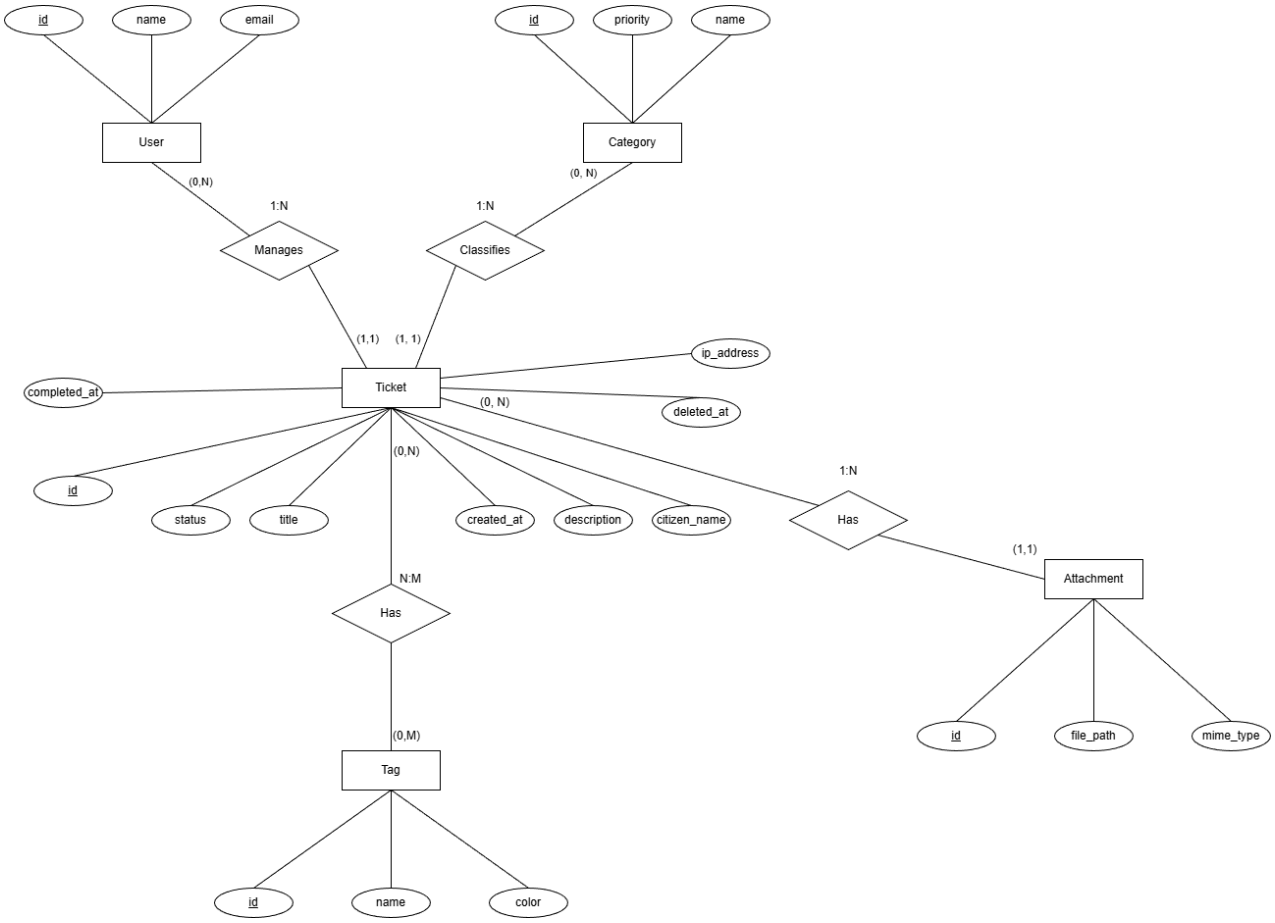
Por último, como funcionalidad añadida, se ha pensado el poder incluir una imagen/foto de la incidencia al ticket .

Los atributos clave serían:

- Id
- Path del archivo
- Tipo de archivo (PNG,JPEG...)

Diagrama de Cheng

He creado el diagrama de Cheng en draw.io que representan las entidades, sus atributos y las relaciones entre ellas.

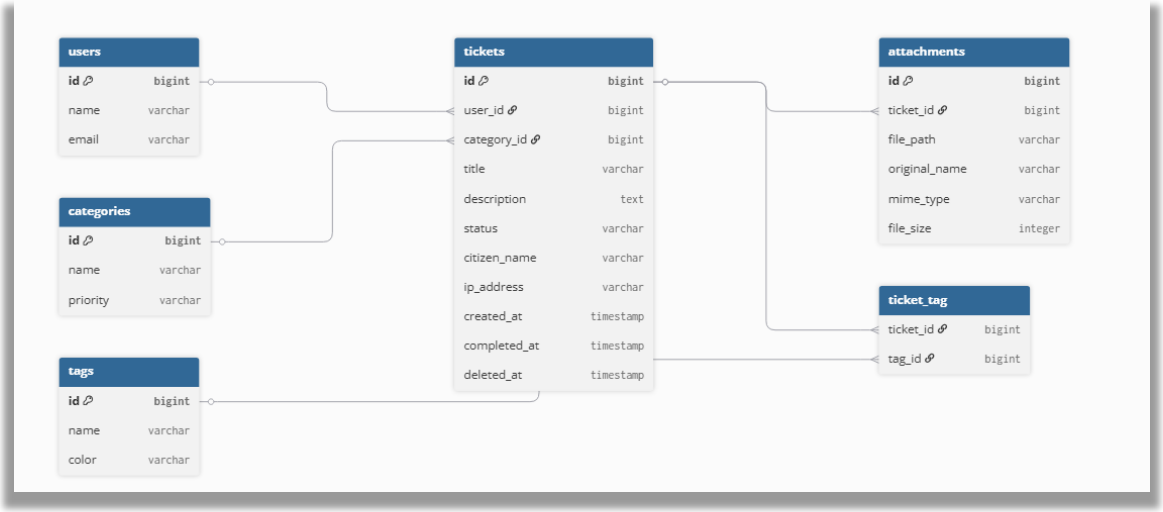


Hago la conversión del modelo E-R al modelo relacional usando dbdiagram.io

El código DBML que he usado para definir el modelo relacional es el siguiente:

```
// 1. Usuarios
Table users {
  id bigint [primary key]
  name varchar
  email varchar
}

// 2. Categorías
Table categories {
  id bigint [primary key]
  name varchar
  priority varchar // 'Alta', 'Media', 'Baja'
}

// 3. Tabla Principal: Tickets
Table tickets {
  id bigint [primary key]
  user_id bigint
  category_id bigint
  title varchar
  description text
  status varchar
  citizen_name varchar
  ip_address varchar
  created_at timestamp
  completed_at timestamp
  deleted_at timestamp
}

// 4. Etiquetas
Table tags {
  id bigint [primary key]
  name varchar
  color varchar
}

```

```
// 5. Tabla Pivote (Relación N:M entre Tickets y Tags)
Table ticket_tag {
  ticket_id bigint
  tag_id bigint
}

// 6. Archivos Adjuntos
Table attachments {
  id bigint [primary key]
  ticket_id bigint
  file_path varchar
  original_name varchar
  mime_type varchar
  file_size integer
}

// --- RELACIONES ---

Ref: tickets.user_id > users.id // Un usuario tiene muchos tickets
Ref: tickets.category_id > categories.id // Una categoría tiene muchos tickets
Ref: attachments.ticket_id > tickets.id // Un ticket tiene muchos adjuntos

// La relación N:M se divide en dos de 1:N hacia la tabla pivote
Ref: ticket_tag.ticket_id > tickets.id
Ref: ticket_tag.tag_id > tags.id

```



Nota: La línea que sale de la tabla tags conecta con el tag_id de la tabla intermedia ticked_tag (para la relación de muchos a muchos), solo que se “mete” por debajo de la tabla tickets y parece que conecta con ésta, pero la línea va por detrás hasta la tabla ticket_tag.

Creación del proyecto en un entorno seguro de codespaces (entorno aislado) y datos “fake” (usando Factories y Seeders)

Al usar Codespaces evito realizar instalaciones de ningún binario en el equipo (PHP, Composer, Node.js, MariaDB), de este modo, el sistema operativo permanece completamente aislado de cualquier error crítico o vulnerabilidad.

Al usar datos sintéticos usando Model Factories de Laravel puedo crear miles de registros ficticios sin usar ningún dato real para esta prueba de concepto.

Creo el repositorio en mi Codespace:

Creo un repositorio vacío que llamo “PoC-Exportation-Project” con la descripción del proyecto:

“PoC: Sistema de Consultas y Exportación Masiva (Laravel 12 + Vue 3). Arquitectura optimizada para memoria (PHP Streams) con soporte universal de caracteres (UTF-8/BOM) y datos anonimizados. Despliegue seguro en Codespaces.”

Instalación limpia de Laravel directamente en la raíz

Pruebo que tengo composer instalado en el repositorio pidiendo la versión.

Composer --version

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ composer --version
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003
.
Composer version 2.9.2 2025-11-19 21:57:25
PHP version 8.3.14 (/usr/local/php/8.3.14/bin/php)
```

Ignorando la advertencia, ya que Xdebug no lo voy a usar ni configurar por el momento, ya puedo ver que tengo instalado Composer (versión 2.9.2) y PHP (versión 8.3.14).

Ahora que he comprobado que tengo Composer y PHP instalados en el sistema, me aseguro de que solo tengo la carpeta oculta .git en la carpeta raíz del proyecto.

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (main) $ ls -la
  total 12
  drwxrwxrwx+ 3 codespace root 4096 Feb 15 07:39 .
  drwxr-xrwx+ 5 codespace root 4096 Feb 15 07:39 ..
  drwxrwxrwx+ 8 codespace root 4096 Feb 15 07:39 .git
```

Como Laravel no me deja instalar un proyecto en la carpeta raíz al tener el .git y no estar vacío (comportamiento proteccionista para no sobrescribir por accidente un proyecto existente) creo el proyecto en una carpeta temporal para luego mover los archivos a la raíz.

```
○ @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (main) $ composer create-project laravel/laravel temp
```

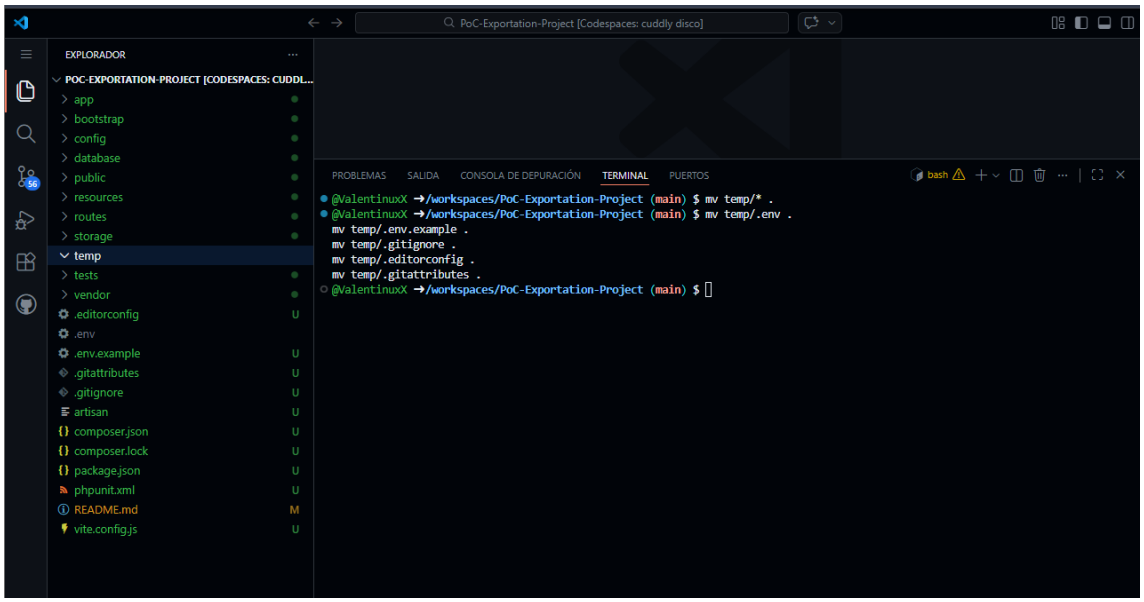
```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ composer create-project laravel/laravel temp
  laravel/tinker ..... DONE
  nesbot/carbon ..... DONE
  nunomaduro/collision ..... DONE
  nunomaduro/termwind ..... DONE

81 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port
).

  INFO  No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
> @php artisan key:generate --ansi
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port
).

  INFO  Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port
).
> @php artisan migrate --graceful --ansi
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port
).

  INFO  Preparing database.

  Creating migration table ..... 132.63ms DONE

  INFO  Running migrations.

  0001_01_01_000000_create_users_table ..... 19.43ms DONE
  0001_01_01_000001_create_cache_table ..... 11.82ms DONE
  0001_01_01_000002_create_jobs_table ..... 13.83ms DONE
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $
```

Muevo los archivos a la carpeta raíz:

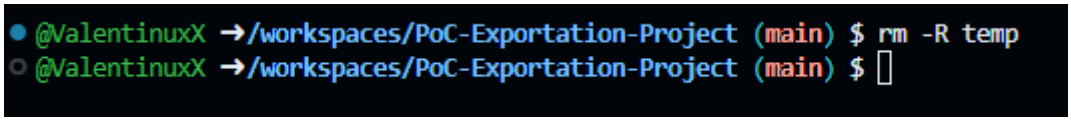
```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ mv temp/* .
```

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ mv temp/.env .
mv temp/.env.example .
mv temp/.gitignore .
mv temp/.editorconfig .
mv temp/.gitattributes .
```

Exceptuando la carpeta temp vacía todo el scaffolding del proyecto está ya en la carpeta raíz:



Por lo que puedo eliminar la carpeta temporal y su contenido:



Actualizo el gitignore

```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS

  GNU nano 7.2                                        .gitignore *
*.log
.DS_Store
.env
.env.backup
.env.production
.phpactor.json
.phpunit.result.cache
/.fleet
/.idea
/.nova
/.phpunit.cache
/.vscode
/.zed
/auth.json
/node_modules
/public/build
/public/hot
/public/storage
/storage/*.key
/storage/pail
/vendor
Homestead.json
Homestead.yaml
Thumbs.db
/storage/framework/cache/data/*
/storage/framework/sessions/*
/storage/framework/testing/*
/storage/framework/views/*
npm-debug.log
yarn-error.log
```

Actualizo el repositorio de git con el scaffolding y creo una rama para desarrollo y pruebas

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git add .
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git commit -m "Estructura base"
```

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git push origin main
  Enumerating objects: 78, done.
  Counting objects: 100% (78/78), done.
  Delta compression using up to 2 threads
  Compressing objects: 100% (59/59), done.
  Writing objects: 100% (76/76), 72.50 KiB | 4.26 MiB/s, done.
  Total 76 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
  remote: Resolving deltas: 100% (1/1), done.
  To https://github.com/ValentinuxX/PoC-Exportation-Project
     c1e987f..516baef  main -> main
```

Ahora que ya tengo la estructura base en la rama main, creo la rama de desarrollo (develop) y me muevo a ella.

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git checkout -b develop
  Switched to a new branch 'develop'
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git push -u origin develop
  Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
  remote:
  remote: Create a pull request for 'develop' on GitHub by visiting:
  remote:      https://github.com/ValentinuxX/PoC-Exportation-Project/pull/new/develop
  remote:
  To https://github.com/ValentinuxX/PoC-Exportation-Project
   * [new branch]      develop -> develop
  branch 'develop' set up to track 'origin/develop'.
```

Instalo e inicializo MariaBD en la MV de Codespaces

Antes de nada actualizo los repositorios:

Sudo apt update

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ sudo apt update
```

Instalo MaridDB:

sudo apt install mariadb-server

E inicio el servicio:

sudo service mariadb start

Creo la base de datos y el usuario para el proyecto

Accedo a MariaDB:

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 35
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Creo la base de datos que llamaré gestor_db:

CREATE DATABASE gestor_db;

Creo el usuario específico para Laravel con la contraseña “secret”

CREATE USER ‘laravel_user’@’localhost’ IDENTIFIED BY ‘1234’;

Le doy permisos al usuario para poder actuar en la base de datos:

GRANT ALL PRIVILEGES ON gestor_db.* TO 'laravel_user'@'localhost';

FLUSH PRIVILEGES;

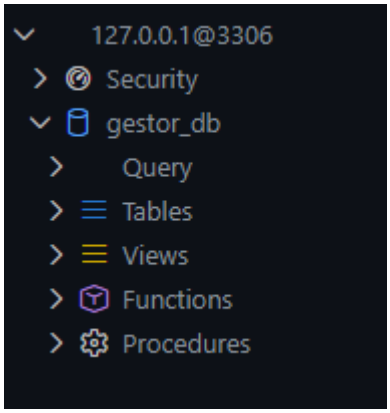
EXIT;

```
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 36
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE gestor_db;
ERROR 1007 (HY000): Can't create database 'gestor_db'; database exists
MariaDB [(none)]> CREATE USER ‘laravel_user’@’localhost’ IDENTIFIED BY ‘1234’;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near ‘‘1234’’ at line 1
MariaDB [(none)]> CREATE USER 'laravel_user'@'localhost' IDENTIFIED BY '1234';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON gestor_db.* TO 'laravel_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> EXIT;
Bye
```

Configuro el archivo .env para que el servidor pueda usar la base de datos

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=gestor_db
DB_USERNAME=laravel_user
DB_PASSWORD=1234
```

Una vez configurada compruebo que puedo conectar con la base de datos:



Se conecta correctamente.

Genero los Models y sus Migrations en Laravel

Ejecuto los comandos para crear los modelos y sus migraciones:

1. Categorías

php artisan make:model Category -m

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan make:model Category -m
  Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).

    INFO  Model [app/Models/Category.php] created successfully.

    INFO  Migration [database/migrations/2026_02_19_095515_create_categories_table.php] created successfully.
```

2. Etiquetas

php artisan make:model Tag -m

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan make:model Tag -m
  Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).

    INFO  Model [app/Models/Tag.php] created successfully.

    INFO  Migration [database/migrations/2026_02_19_095826_create_tags_table.php] created successfully.
```

3. Tickets (La tabla central)

php artisan make:model Ticket -m

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan make:model Ticket -m
  Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).

    INFO  Model [app/Models/Ticket.php] created successfully.

    INFO  Migration [database/migrations/2026_02_19_110415_create_tickets_table.php] created successfully.
```

4. Tabla Pivote (Relación N:M) - Solo migración, no necesita modelo

php artisan make:migration create_ticket_tag_table

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan make:migration create_ticket_tag_table
  Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).

    INFO  Migration [database/migrations/2026_02_19_110620_create_ticket_tag_table.php] created successfully.
```

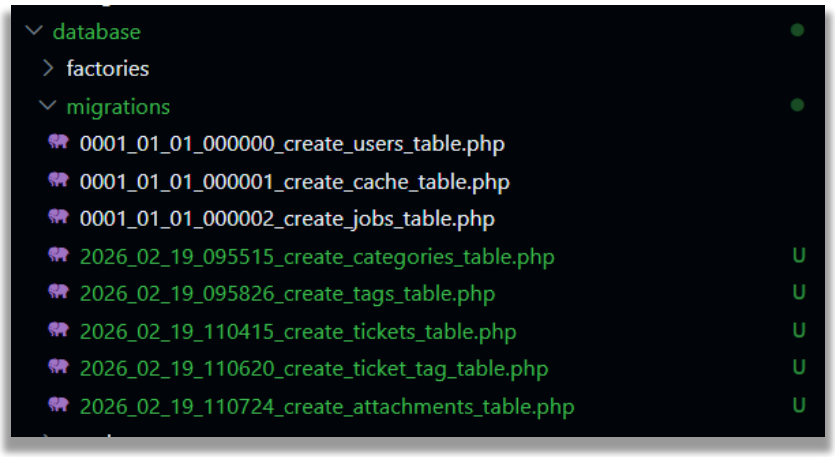
5. Archivos Adjuntos

php artisan make:model Attachment -m

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan make:model Attachment -m
  Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).

    INFO  Model [app/Models/Attachment.php] created successfully.

    INFO  Migration [database/migrations/2026_02_19_110724_create_attachments_table.php] created successfully.
```

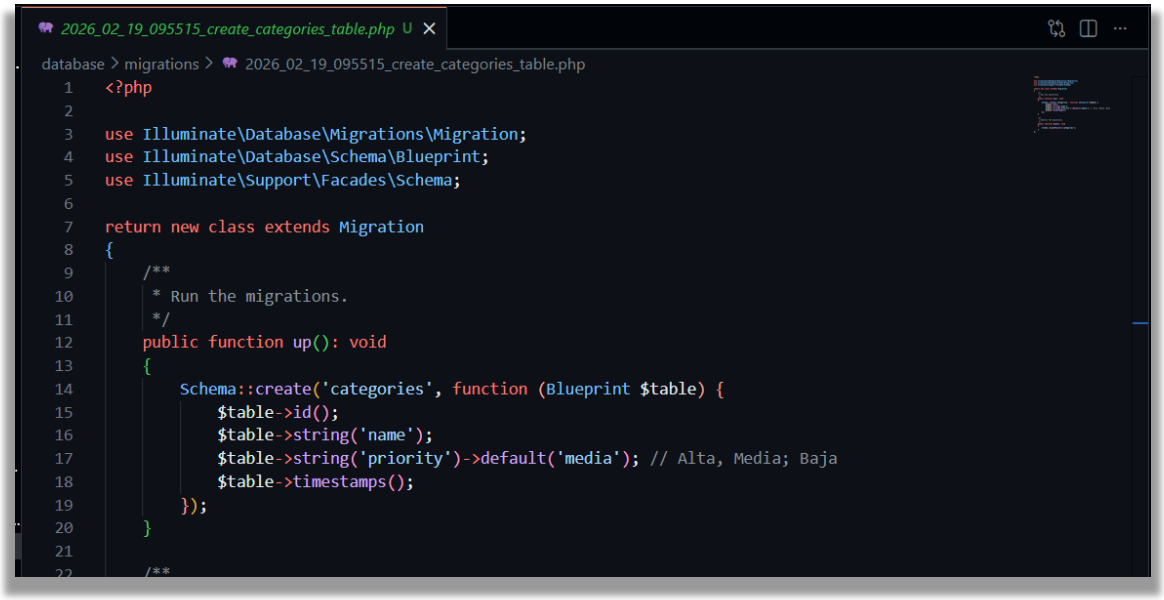
Compruebo que se han creado correctamente tanto los modelos como sus migraciones:

```
> app
  > Http
  v Models
    Attachment.php   U
    Category.php     U
    Tag.php          U
    Ticket.php       U
    User.php
```



Una vez que tengo creados los modelos con sus migraciones edito las migraciones para que esté en concordancia con mi diseño de base de datos

Edito en la migración de la tabla categorías (categories) la función up():



2026_02_19_095515_create_categories_table.php U

database > migrations > 2026_02_19_095515_create_categories_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('priority')->default('media'); // Alta, Media; Baja
            $table->timestamps();
        });
    }

    /**
```

La función up de la migración de la tabla de etiquetas (tags)

2026_02_19_095826_create_tags_table.php U

database > migrations > 2026_02_19_095826_create_tags_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('tags', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('color')->nullable(); //puede ser nulo o con color (ej: #FF0000)
            $table->timestamps();
        });
    }

```

Edito la función up() para la migración que va a contener.

2026_02_19_110415_create_tickets_table.php

database > migrations > 2026_02_19_110415_create_tickets_table.php

```php
{
    /**
     */
    public function up(): void
    {
        Schema::create('tickets', function (Blueprint $table) {
            $table->id();
            // Claves foráneas
            $table->foreignId('user_id')->constrained();
            $table->foreignId('category_id')->constrained();

            // Datos
            $table->string('title');
            $table->text('description');
            $table->string('status')->default('Pendiente');
            $table->string('citizen_name')->nullable();
            $table->ipAddress('ip_address')->nullable();

            //Fechas y deletes
            $table->timestamp('completed_at')->nullable();
            $table->softDeletes(); // Crea el campo deleted_at automáticamente
            $table->timestamps();
        });
    }

```

Edito la función up() de la migración de la tabla intermedia para la relación de muchos a muchos de tags y tickets.

2026_02_19_110620_create_ticket_tag_table.php

database > migrations > 2026_02_19_110620_create_ticket_tag_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('ticket_tag', function (Blueprint $table) {
            $table->id();
            // cascadeOnDelete hace que si se borra un ticket, se borre su relación con la etiqueta
            $table->foreignId('ticket_id')->constrained()->cascadeOnDelete();
            $table->foreignId('tag_id')->constrained()->cascadeOnDelete();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
```

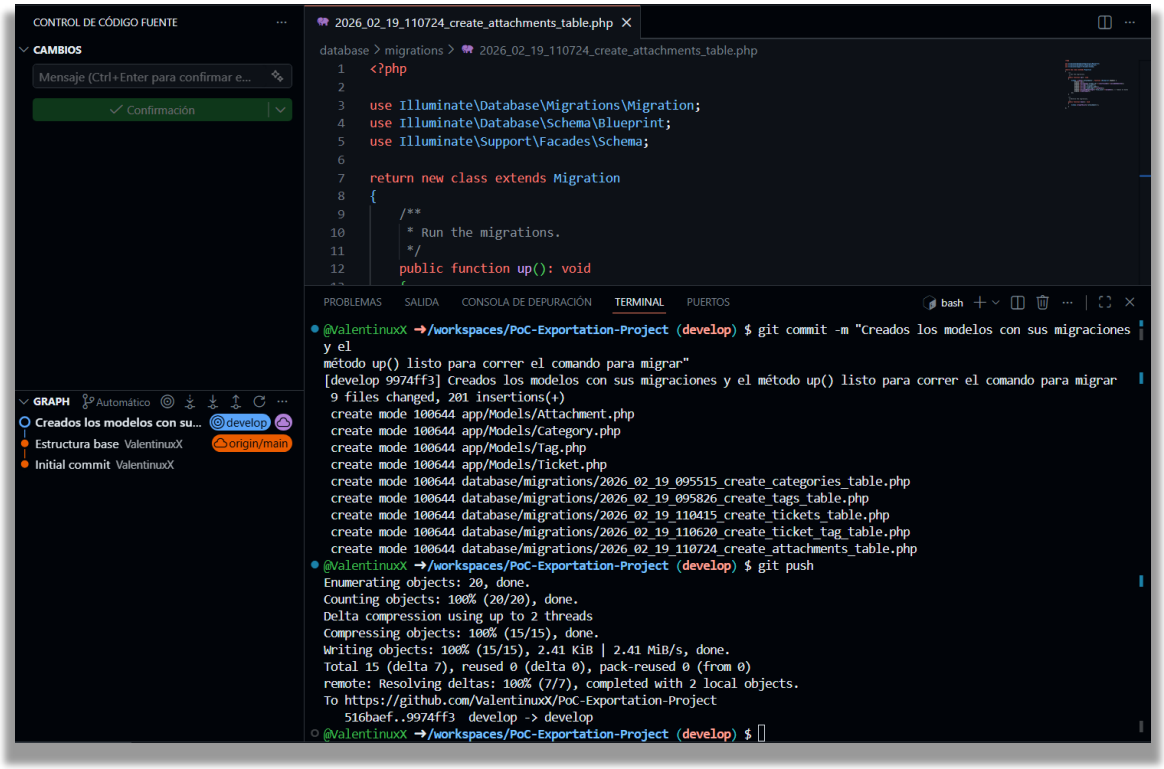
Por último, edito la función up() de la migración de la tabla de los Attachments:

2026_02_19_110724_create_attachments_table.php

database > migrations > 2026_02_19_110724_create_attachments_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('attachments', function (Blueprint $table) {
            $table->id();
            $table->foreignId('ticket_id')->constrained()->cascadeOndelete();
            $table->string('file_path');
            $table->string('original_name');
            $table->string('mime_type')->nulable();
            $table->unsignedBigInteger('file_size')->nulable(); // Tamaño en bytes
            $table->timestamps();
        });
    }

    /**
```

Hago un commit y actualizo en el repositorio remoto antes de realizar las migraciones



Realizo las migraciones una vez que las tengo ya preparadas

Una vez que ya he preparado mis migraciones actualizando el método up() ejecuto las migraciones con el comando:

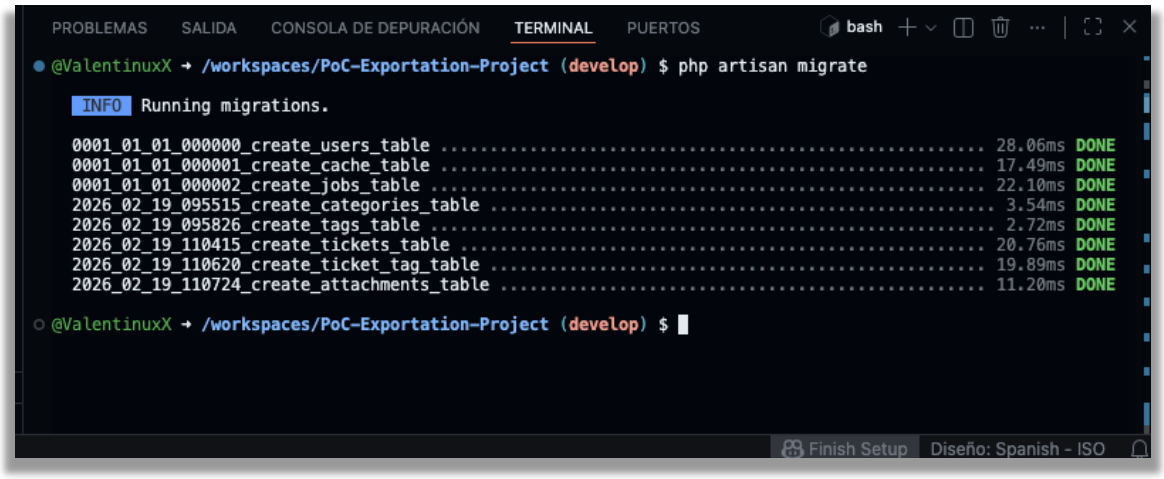
/usr/bin/php8.3 artisan migrate

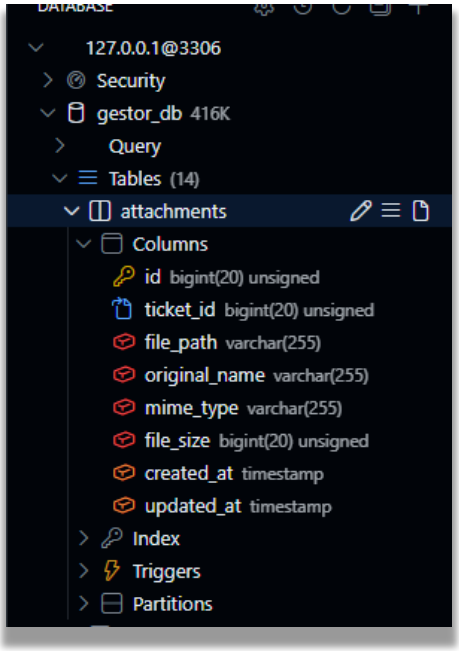
Nota: por culpa del php de Codespace me he visto obligado a instalar otra versión de php que se encuentra en /usr/bin/php8.3, por lo que para indicarle que quiero usar ese php que me funciona en el entorno de Linux de Codespaces, y que me haga las migraciones, he tenido que usar el comando tal como he indicado arriba.

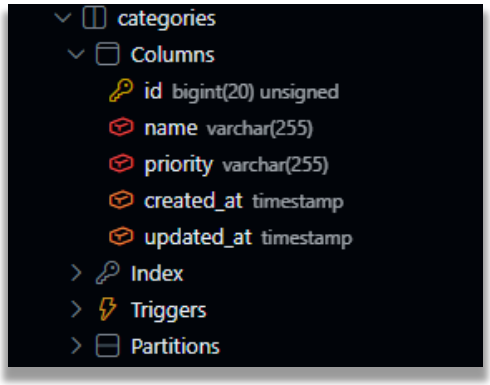
Por eso me he creado un alias para que al escribir php en la terminal la terminal interprete /usr/bin/php8.3, he hecho un migration:rollback para “tirar” las migraciones un paso a tras y lo he vuelto a hacer con el nuevo alias.

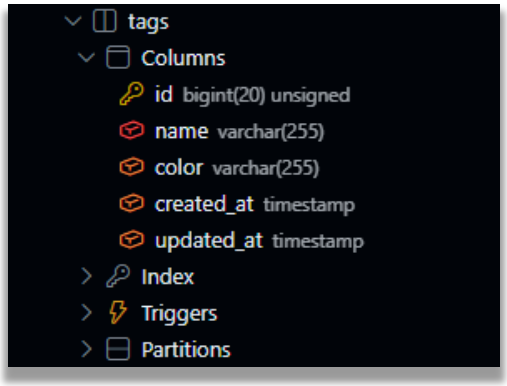
```
echo "alias php='/usr/bin/php8.3'" >> ~/.bashrc

source ~/.bashrc
```

Con el nuevo alias y lo he podido hacer con el comando tal como se usaría:




DATABASE
127.0.0.1@3306
Security
gestor_db 416K
Query
Tables (14)
attachments
Columns
id bigint(20) unsigned
ticket_id bigint(20) unsigned
file_path varchar(255)
original_name varchar(255)
mime_type varchar(255)
file_size bigint(20) unsigned
created_at timestamp
updated_at timestamp
Index
Triggers
Partitions


categories
Columns
id bigint(20) unsigned
name varchar(255)
priority varchar(255)
created_at timestamp
updated_at timestamp
Index
Triggers
Partitions


tags
Columns
id bigint(20) unsigned
name varchar(255)
color varchar(255)
created_at timestamp
updated_at timestamp
Index
Triggers
Partitions

Preparo los modelos para que el ORM (Eloquent) se pueda comunicar con la base de datos

Ahora que ya tengo las tablas en la base de datos preparo los modelos para que el ORM que utiliza Laravel pueda comunicarse con la base de datos. Le indico los campos rellenables (los fillable) y el método en el que indico las relaciones.

En el modelo Category:

Tag.php M

app > Models > Tag.php > ...

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Tag extends Model
{
    protected $fillable = ['name', 'color'];

    public function tickets()
    {
        return $this->belongsToMany(Ticket::class);
    }

}
```

Attachment.php M

app > Models > Attachment.php > ...

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Attachment extends Model
{
    protected $fillable = [
        'ticket_id', 'file_path', 'original_name', 'mime_type', 'file_size'
    ];

    public function ticket()
    {
        return $this->belongsTo(Ticket::class);
    }
}
```

Ticket.php M

app > Models > Ticket.php > Ticket > attachments

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Ticket extends Model
{
    protected $fillable = [
        'user_id', 'category_id', 'title', 'description',
        'status', 'citizen_name', 'ip_address', 'completed_at'
    ];

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function tags()
    {
        return $this->belongsToMany(Tag::class);
    }

    public function attachments()
    {
        return $this->hasMany(Attachment::class);
    }
}
```

User.php M

app > Models > User.php > User > tickets

```php
class User extends Authenticatable
    *
    * @var list<string>
    */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
    * Get the attributes that should be cast.
    *
    * @return array<string, string>
    */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }

    public function tickets()
    {
        return $this->hasMany(Ticket::class);
    }
}
```

Uso Tinker para probar lo desarrollado hasta ahora antes de hacer un commit y seguir adelante con el desarrollo

Entro en Tinker desde la consola:

php artisan tinker

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan tinker
Psy Shell v0.12.20 (PHP 8.3.6 — cli) by Justin Hileman
New PHP manual is available (latest: 3.0.2). Update with `doc --update-manual`
>
```

Creo el usuario administrador (yo):

$user = App\Models\User::create(['name' => 'Valentín Fernández', 'email' => 'admin@test.com', 'password' => bcrypt('123456')]);

```
> $user = App\Models\User::create(['name' => 'Valentín Fernández', 'email' => 'admin@test.com', 'password' => bcrypt('123456')]);
= App\Models\User {#5839
    name: "Valentín Fernández",
    email: "admin@test.com",
    #password: "$2y$12$7pdaNYyW3kK5rNBy8Fxn3.HrZECjM05zhhdttcvKumqtdTBIPwD1K",
    updated_at: "2026-02-21 06:38:47",
    created_at: "2026-02-21 06:38:47",
    id: 1,
  }

>
```

Creo una primera categoría:

$category = App\Models\Category::create(['name' => 'Fallo del sistema', 'priority' => 'Alta']);

```
> $category = App\Models\Category::create(['name' => 'Fallo del sistema', 'priority' => 'Alta']);
= App\Models\Category {#6585
    name: "Fallo del sistema",
    priority: "Alta",
    updated_at: "2026-02-21 06:54:19",
    created_at: "2026-02-21 06:54:19",
    id: 1,
  }

>
```

Y, por último, pruebo a crear un primer ticket:

$ticket = App\Models\Ticket::create([

'user_id' => $user->id,

'category_id' => $category->id,

'title' => 'El servidor se ha caído',

'description' => 'El servidor no da respuesta y no puedo hacer una consulta.',

'satus' => 'Pendiente'

]);

```
> $ticket = App\Models\Ticket::create([
. 'user_id' => $user->id,
. 'category_id' => $category->id,
. 'title' => 'El servidor se ha caído',
. 'description' => 'El servidor no da respuesta y no puedo hacer una consulta.',
. 'satus' => 'Pendiente'
. ]);
= App\Models\Ticket {#6650
    user_id: 1,
    category_id: 1,
    title: "El servidor se ha caído",
    description: "El servidor no da respuesta y no puedo hacer una consulta.",
    updated_at: "2026-02-21 07:00:23",
    created_at: "2026-02-21 07:00:23",
    id: 1,
  }

>
```

Nota: He cometido un error al crear el ticket y en la palabra estatus me he comido una t, pero como en mi sistema precisamente puse que por defecto el estado sea “Pendiente” no veo necesario cambiarlo.

Empiezo a construir la API para que luego Vue pueda conectarse con mi Backend

En mi versión de Laravel, por defecto vienen las rutas de API apagadas, por lo que el primer paso es instalarlas:

php artisan install:api

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan install:api
PHP Warning:  PHP Startup: Unable to load dynamic library 'pdo_mysql' (tried: /usr/local/php/8.3.14/extensions/pdo_mysql (/usr/local/php
/8.3.14/extensions/pdo_mysql: cannot open shared object file: No such file or directory), /usr/local/php/8.3.14/extensions/pdo_mysql.so
(/usr/local/php/8.3.14/extensions/pdo_mysql.so: cannot open shared object file: No such file or directory)) in Unknown on line 0
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).
PHP Warning:  PHP Startup: Unable to load dynamic library 'pdo_mysql' (tried: /usr/local/php/8.3.14/extensions/pdo_mysql (/usr/local/php
/8.3.14/extensions/pdo_mysql: cannot open shared object file: No such file or directory), /usr/local/php/8.3.14/extensions/pdo_mysql.so
(/usr/local/php/8.3.14/extensions/pdo_mysql.so: cannot open shared object file: No such file or directory)) in Unknown on line 0
./composer.json has been updated
Running composer update laravel/sanctum
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking laravel/sanctum (v4.3.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Downloading laravel/sanctum (v4.3.1)
  - Installing laravel/sanctum (v4.3.1): Extracting archive
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi
PHP Warning:  PHP Startup: Unable to load dynamic library 'pdo_mysql' (tried: /usr/local/php/8.3.14/extensions/pdo_mysql (/usr/local/php
/8.3.14/extensions/pdo_mysql: cannot open shared object file: No such file or directory), /usr/local/php/8.3.14/extensions/pdo_mysql.so
(/usr/local/php/8.3.14/extensions/pdo_mysql.so: cannot open shared object file: No such file or directory)) in Unknown on line 0
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).

  INFO  Discovering packages.

  laravel/pail ........................................................ DONE
  laravel/sail ........................................................ DONE
  laravel/sanctum ..................................................... DONE
  laravel/tinker ...................................................... DONE
  nesbot/carbon ....................................................... DONE
  nunomaduro/collision ................................................ DONE
  nunomaduro/termwind ................................................. DONE

81 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force
PHP Warning:  PHP Startup: Unable to load dynamic library 'pdo_mysql' (tried: /usr/local/php/8.3.14/extensions/pdo_mysql (/usr/local/php
/8.3.14/extensions/pdo_mysql: cannot open shared object file: No such file or directory), /usr/local/php/8.3.14/extensions/pdo_mysql.so
(/usr/local/php/8.3.14/extensions/pdo_mysql.so: cannot open shared object file: No such file or directory)) in Unknown on line 0
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).

  INFO  No publishable resources for tag [laravel-assets].
```

```
No security vulnerability advisories found.

  INFO  Published API routes file.

 One new database migration has been published. Would you like to run all pending database migrations? (yes/no) [yes]:
 > yes

  INFO  Running migrations.

  2026_02_21_071513_create_personal_access_tokens_table ............................................................ 26.34ms DONE


  INFO  API scaffolding installed. Please add the [Laravel\Sanctum\HasApiTokens] trait to your User model.

○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

Ahora que veo que mi Backend funciona probándolo con Tinker y haber instalado ya la funcionalidad de API en Laravel creo que es un buen momento de hacer un commit y hacer un push al repositorio remoto

Añado todos los cambios:

git add .

Creo el commit:

git commit -m “Backend funcionando e instalada la funcionalidad API en Laravel”

Hago el push:

git push

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git add .
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git commit -m "Backend funcionando e instalada la funcionalidad de API en
  Laravel"
  [develop f8ccd31] Backend funcionando e instalada la funcionalidad de API en Laravel
   11 files changed, 241 insertions(+), 5 deletions(-)
   create mode 100644 config/sanctum.php
   create mode 100644 database/migrations/2026_02_21_071513_create_personal_access_tokens_table.php
   create mode 100644 routes/api.php
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git push
  Enumerating objects: 36, done.
  Counting objects: 100% (36/36), done.
  Delta compression using up to 2 threads
  Compressing objects: 100% (20/20), done.
  Writing objects: 100% (20/20), 4.18 KiB | 4.18 MiB/s, done.
  Total 20 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
  remote: Resolving deltas: 100% (9/9), completed with 8 local objects.
  To https://github.com/ValentinuxX/PoC-Exportation-Project
     5d18e21..f8ccd31  develop -> develop
○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

Empiezo a crear los controladores

Creo el controlador para los tickets usando la bandera –api para que Laravel cree las 5 funciones básicas (el CRUD):

php artisan make:controller Api/TicketController –api

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan make:controller Api/TicketController --api
  INFO  Controller [app/Http/Controllers/Api/TicketController.php] created successfully.
```

Sigo escribiendo la lógica del controlador. Indico que use el modelo Ticket y modifico el método index() para que me traiga (en formato json) todos los tickets, pero añadiendo en la misma consulta los datos de usuario y categoría:

```
TicketController.php U ×
app > Http > Controllers > Api > TicketController.php > TicketController > show
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use App\Models\Ticket;
7  use Illuminate\Http\Request;
8
9  class TicketController extends Controller
10 {
11     /**
12      * Display a listing of the resource.
13      */
14     public function index()
15     {
16         // Traigo todos los tickets incuyendo los datos del usuario y la categoría en la misma consulta
17         $tickets = Ticket::with(['user', 'category'])->get();
18
19         return response()->json($tickets);
20     }
21
22     /**
23      * Store a newly created resource in storage.
24      */
25     public function store(Request $request)
26     {
27         //
28     }
29
30     /**
```

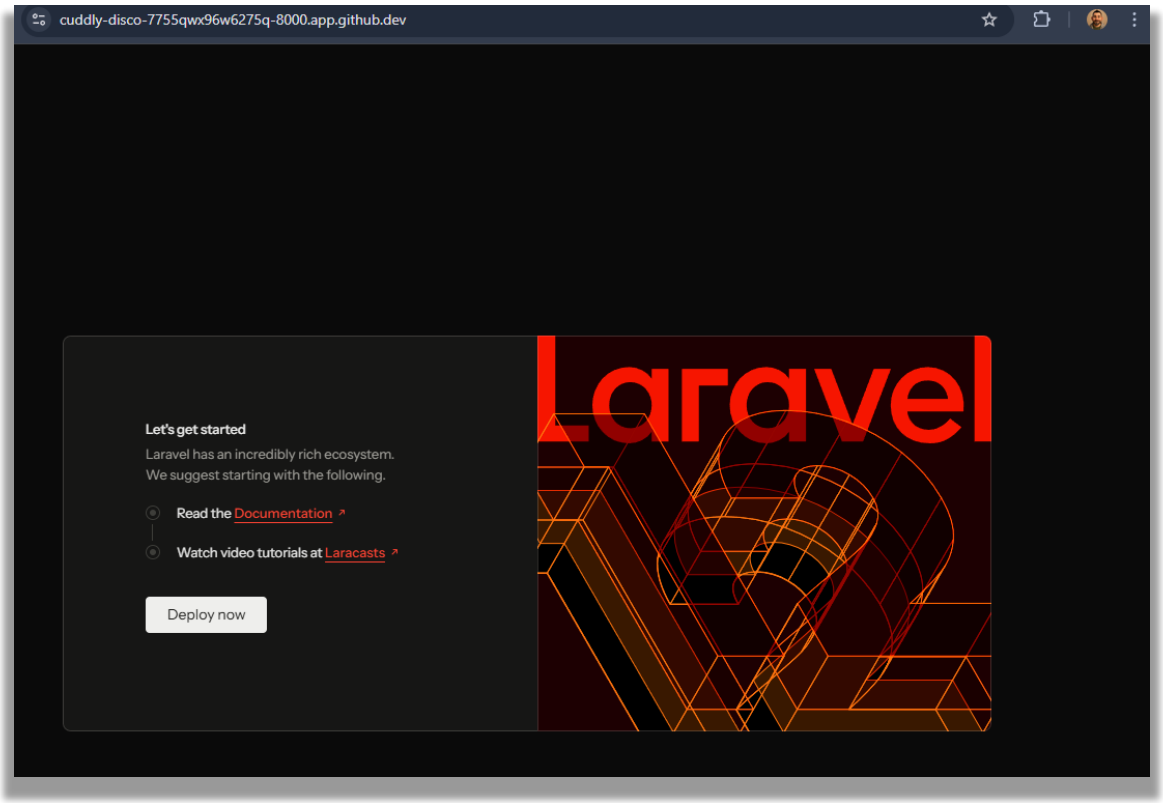
Una vez creado el controlador creo mi primer Endpoint exponiendo la ruta:

```php
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\TicketController; // importo el controlador

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

// Creo la ruta para la API de tickets
Route::apiResource('tickets', TicketController::class);
```

Corro por primera vez el servidor

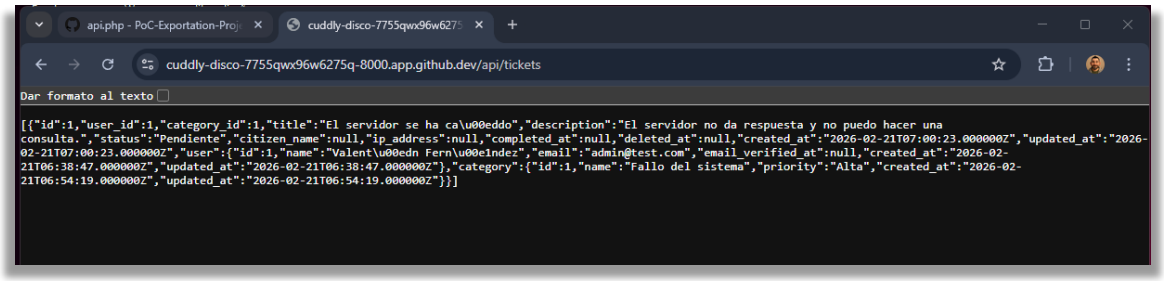
Una vez que tengo el circuito cerrado ya puedo exponer mi API corriendo por primera vez el servidor.

php artisan serve



Que pueda ver la pantalla de inicio de Laravel significa que tengo mi servidor corriendo y funcionando. Pero mis rutas son una API, por lo que mis rutas, realmente están en en las rutas /api, es decir, que mi ruta tickets estaría en /api/tickets.

Si añado /api/tickets (al hacerme el get) tendría que traerme todos los tickets en formato json.



Salvando que el estándar de seguridad de json no me muestra las palabras con tilde y que no será ningún problema cuando lo “coja” el Frontend ya tengo mi API expuesta y funcionando.

Ahora que ya veo que puedo traer los tickets desde la API, es el momento de implementar la funcionalidad de almacenar tickets desde fuera.

Para ello lo haré teniendo en cuenta hacer un fuerte control de la seguridad en la entrada con validaciones.

Implemento la lógica de validar y almacenar un ticket en la base de datos en el controlador:

```
TicketController.php U
app > Http > Controllers > Api > TicketController.php > TicketController > show
9   class TicketController extends Controller
23      * Store a newly created resource in storage.
24      */
25      public function store(Request $request)
26      {
27          // Valido los datos de entrada
28          $validated = $request->validate([
29              'title' => 'required|string|max:255',
30              'description' => 'required|string',
31              'category_id' => 'required|exists:categories,id', // Debe existir
32              'user_id' => 'required|exists:users,id',
33              'status' => 'nullable|string'
34          ]);
35
36          // Crear el ticket (usando el $fillable del modelo)
37          $ticket = Ticket::create($validated);
38
39          // Devuelvo el ticket recién creado con un código de estado HTTP 201
40          return response()->json($ticket, 201);
41      }
42
43      /**
44       * Display the specified resource.
45       */
```

Antes de iniciar la fase de testing hago un commit

```
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ git add .
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ git commit -m "API creando y guardando tickets antes de
hacer el testeo"
[develop eaedefe] API creando y guardando tickets antes de hacer el testeo
 2 files changed, 72 insertions(+)
 create mode 100644 app/Http/Controllers/Api/TicketController.php
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ git push
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 2 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 1.59 KiB | 1.59 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ValentinuxX/PoC-Exportation-Project
   f8ccd31..eaedefe  develop -> develop
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $
```

Comienzo la parte de testing de la API

Esta etapa de testing la voy a dividir en dos fases. Una va a ser hacer un test usando Postman de traer y postear tickets y comprobar que la API no se rompe al tratar de postear con datos que no son correctos.

La otra etapa va a ser crear varios test automatizados. Prerpararé la base de datos de prueba usando SQLite para los test. Así los test correrán en milisegundos y no ensuciarán los datos reales.

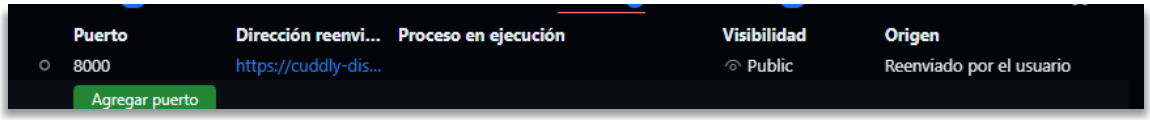
Compruebo que Postman se conecta bien con mi servidor en codespaces.

Como mi servidor no está en mi localhost sino en un Codespaces, el puerto 8000 se encuentra privado, por lo que solo yo con mi navegador (porque estoy logueado en Github) puedo ver los datos.

Si intentara ahora enviar una desde Postman Github le devolvería un HTML pidiendo que inicie sesión.

Soluciono esto:

En VS Code en la pestaña (Ports) debo hacer clic derecho en visibilidad del puerto y lo cambio a público.



Ahora la base real para Postman será:

https://cuddly-disco-7755qwx96w6275q-8000.app.github.dev/api/tickets

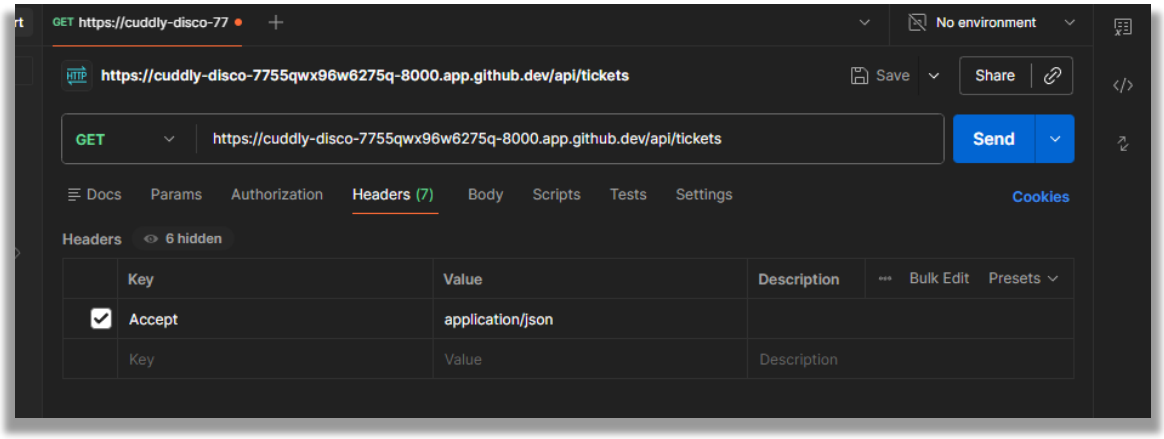
Y los Headers añado las reglas para las APIs de Laravel:

Key: Accept

Value: application/json (pase lo que pase Laravel devolverá un json, nunca un HTML de error).

Ahora voy a lanzar las 3 pruebas planeadas usando la URL:

Prueba GET:



Le hago la petición:

GET https://cuddly-disco-77

https://cuddly-disco-7755qwx96w6275q-8000.app.github.dev/api/tickets

GET https://cuddly-disco-7755qwx96w6275q-8000.app.github.dev/api/tickets

Docs Params Authorization Headers (8) Body Scripts Tests Settings

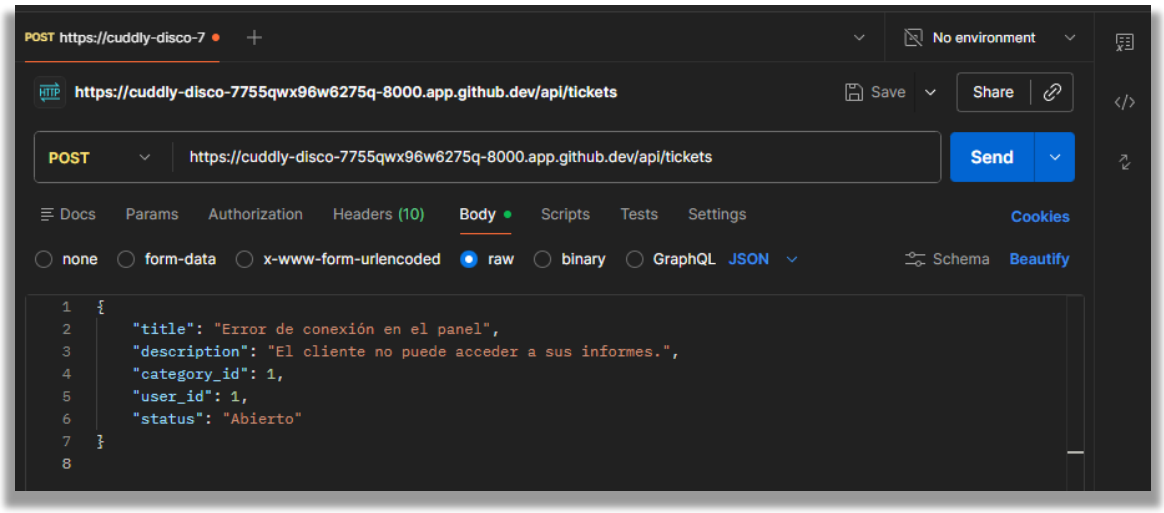
Body Cookies (1) Headers (26) Test Results

{} JSON Preview Visualize

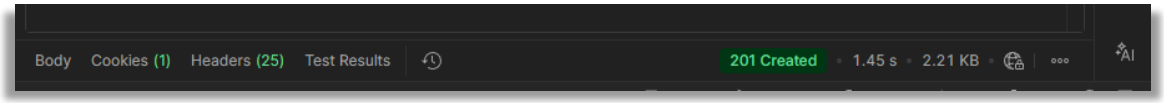

```
[
    {
        "id": 1,
        "user_id": 1,
        "category_id": 1,
        "title": "El servidor se ha caído",
        "description": "El servidor no da respuesta y no puedo hacer una consulta.",
        "status": "Pendiente",
        "citizen_name": null,
        "ip_address": null,
        "completed_at": null,
        "deleted_at": null,
        "created_at": "2026-02-21T07:00:23.000000Z",
        "updated_at": "2026-02-21T07:00:23.000000Z",
        "user": {
            "id": 1,
            "name": "Valentín Fernández",
            "email": "admin@test.com",
            "email_verified_at": null,
            "created_at": "2026-02-21T06:38:47.000000Z",
            "updated_at": "2026-02-21T06:38:47.000000Z"
        },
        "category": {
            "id": 1,
            "name": "Fallo del sistema",
            "priority": "Alta",
            "created_at": "2026-02-21T06:54:19.000000Z",
            "updated_at": "2026-02-21T06:54:19.000000Z"
        }
    }
]
```


200 OK • 625 ms • 2.65 KB

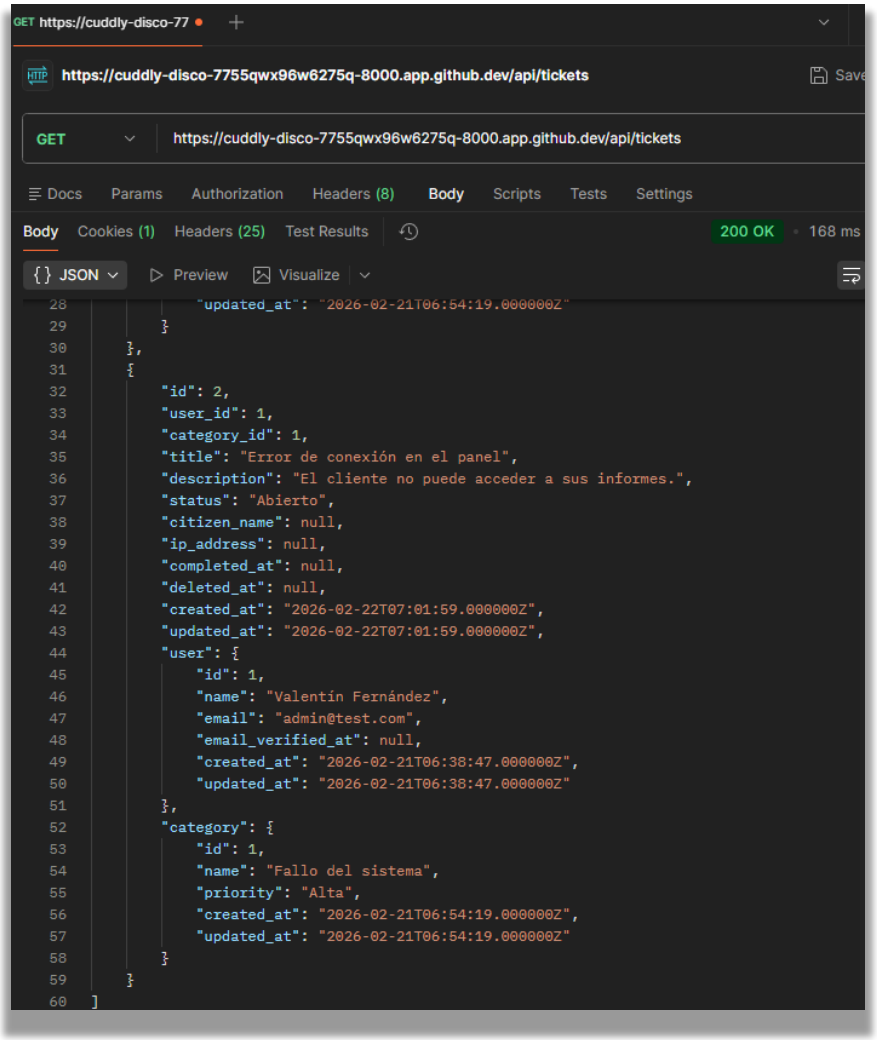
Hago la prueba POST:



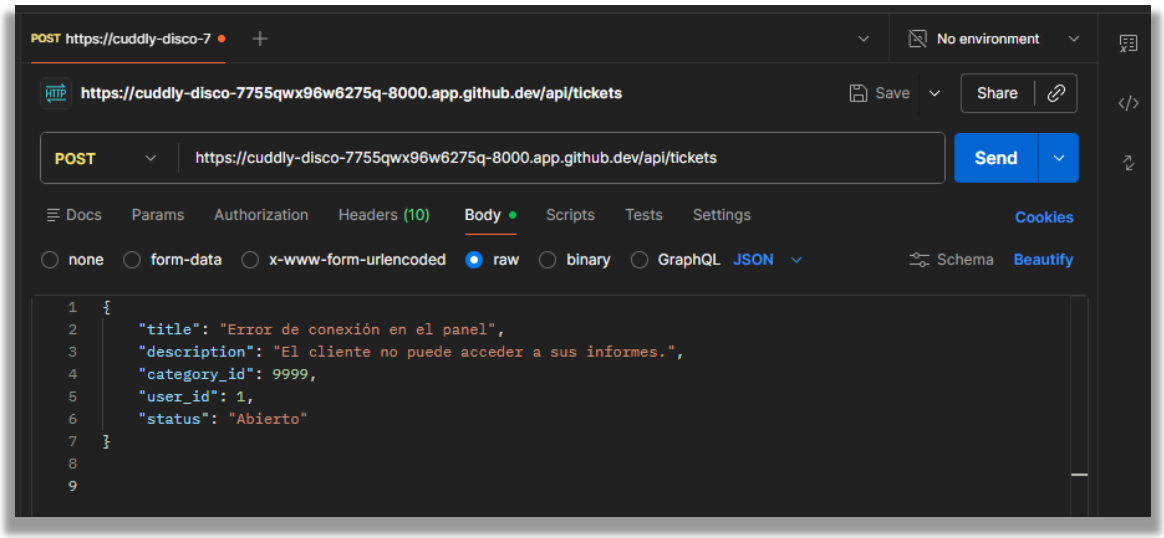
Al enviarlo me ha devuelto un 201 Created:



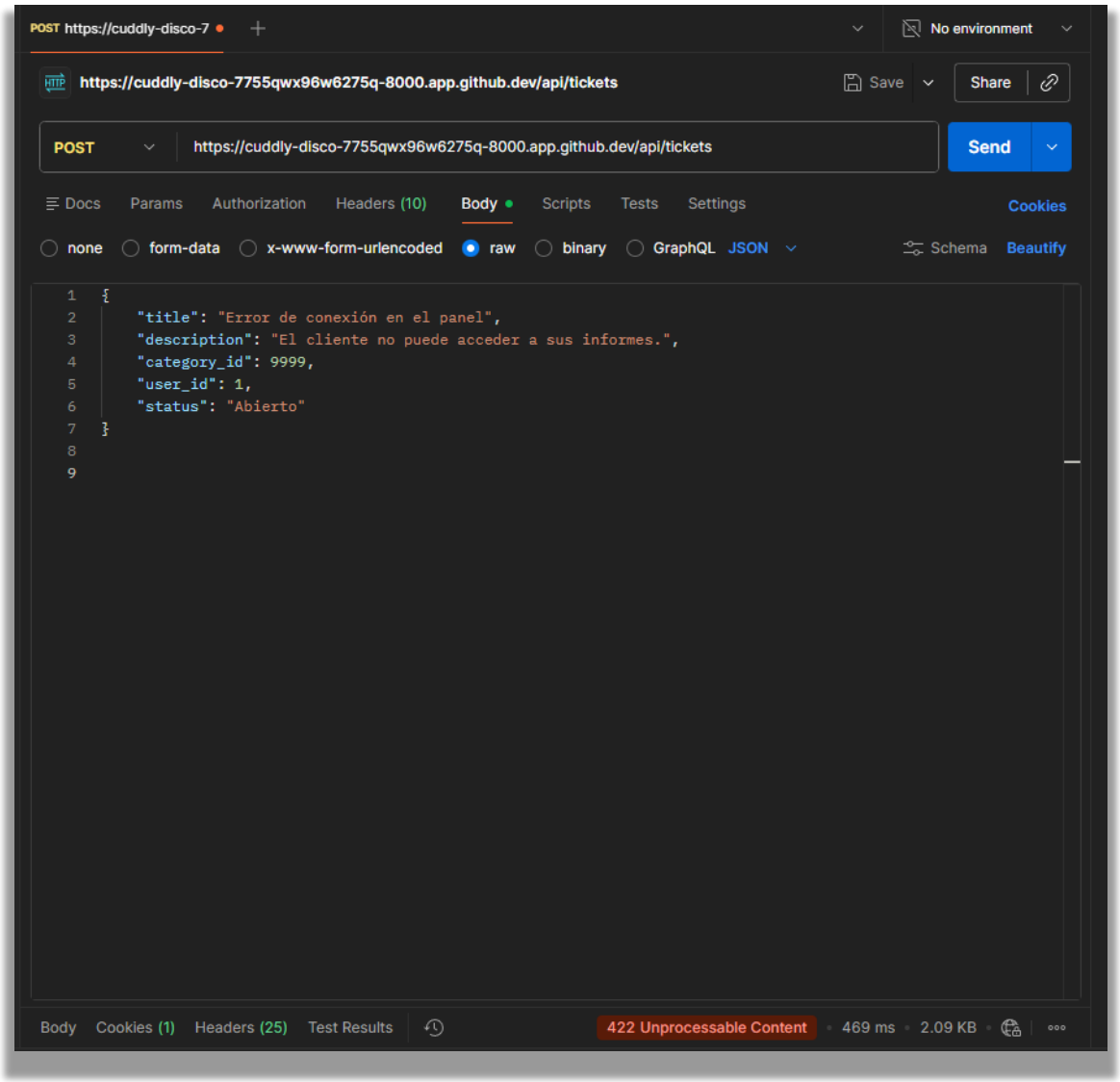
Compruebo que al volver a hacer un GET me devuelve el json con el nuevo ticket:



Sigo con la última prueba, intentar un POST con datos inválidos. Para probarlo lo hago cambiando la categoría por el id 9999:



Me devuelve un 422 que era la respuesta esperada:



Hago una última prueba cambiando el usuario al id 2:

POST https://cuddly-disco-7

No environment

https://cuddly-disco-7755qwx96w6275q-8000.app.github.dev/api/tickets

Save | Share

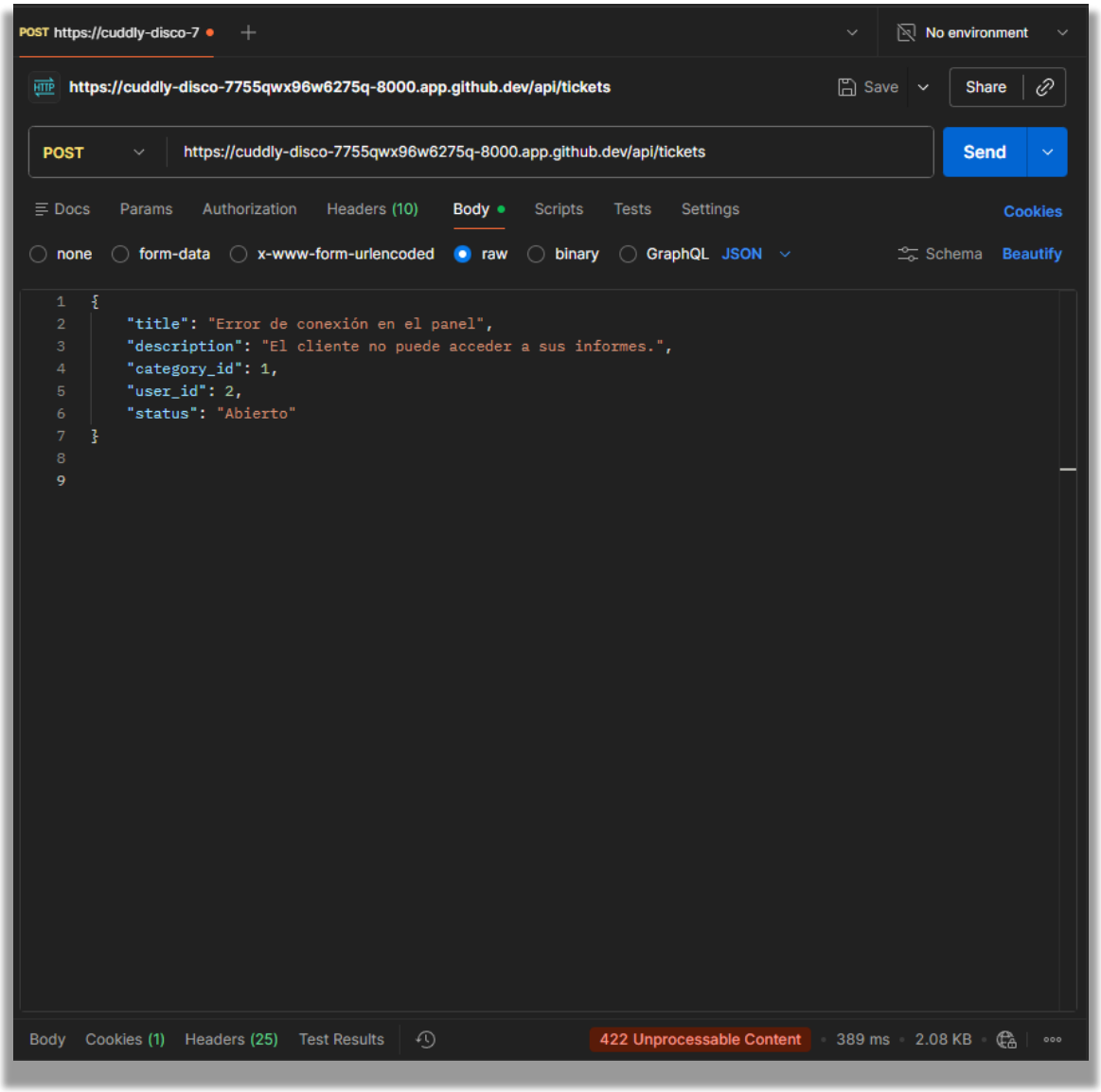
POST https://cuddly-disco-7755qwx96w6275q-8000.app.github.dev/api/tickets Send

Docs | Params | Authorization | Headers (10) | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1 {
2     "title": "Error de conexión en el panel",
3     "description": "El cliente no puede acceder a sus informes.",
4     "category_id": 1,
5     "user_id": 2,
6     "status": "Abierto"
7 }
8
9
```

Me devuelve el 422:



Comienzo la fase de los test automatizados

Para ello voy a dar dos pasos iniciales. Uno va a ser aislar la base de datos y, el otro, crear el archivo de test.

Como no quiero que mi base de datos MariaDB y que la idea es usar una base de datos de usar y tirar en la RAM para que se destruyan después usaré SQLite.

Para ello modfico el archivo phpunit.xml:

Descomento las líneas, si están comentadas, que indican que use SQLite como motor y que los datos los guarde en la RAM (en mi versión de Laravel, la 12, ya aprecen descomentadas):

```
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

Ahora creo el archivo de Test:

php artisan make:test Api/TicketApiTest

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan make:test Api/TicketApiTest

  INFO  Test [tests/Feature/Api/TicketApiTest.php] created successfully.

○ @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $
```

Ahora programo las pruebas automatizadas usando PHP:

TicketApiTest.php U

tests > Feature > Api > TicketApiTest.php > TicketApiTest > test_el_escudo_bloquea_tickets_con_categoria_inventada

```php
<?php

namespace Tests\Feature\Api;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;
use App\Models\User;
use App\Models\Category;
use App\Models\Ticket;

class TicketApiTest extends TestCase
{

    // Antes de cada test, ejecuta las migraciones en la memoria RAM
    // y cuando termine el test, borra todo para dejarla limpia".
    use RefreshDatabase;

    public function test_puede_obtener_todos_los_tickets()
    {
        // 1. PREPARACIÓN (Arrange): Crea datos falsos en la RAM
        $user = User::create(['name' => 'Test User', 'email' => 'test@test.com', 'password' => '123456']);
        $category = Category::create(['name' => 'Fallo', 'priority' => 'Alta']);
        Ticket::create([
            'title' => 'Ticket de prueba GET',
            'description' => 'Descripción de prueba',
            'category_id' => $category->id,
            'user_id' => $user->id,
            'status' => 'Pendiente'
        ]);

        // 2. ACCIÓN (Act): El robot hace un GET a mi API (como en Postman)
        $response = $this->getJson('/api/tickets');

        // 3. VERIFICACIÓN (Assert): Comprobamos que devuelve un 200 OK y trae el ticket
        $response->assertStatus(200)
                 ->assertJsonCount(1) // Comprueba que hay 1 ticket en la lista
                 ->assertJsonFragment(['title' => 'Ticket de prueba GET']);
    }

    public function test_puede_crear_un_ticket_valido()
    {
        // 1. PREPARACIÓN
        $user = User::create(['name' => 'Test User', 'email' => 'test2@test.com', 'password' => '123456']);
        $category = Category::create(['name' => 'Duda', 'priority' => 'Baja']);

        $datosPayload = [
            'title' => 'Ticket desde el Test POST',
            'description' => 'Validando el insert automático',
            'category_id' => $category->id,
            'user_id' => $user->id,
            'status' => 'Abierto'
        ];

        // 2. ACCIÓN: El robot hace un POST enviando el JSON
        $response = $this->postJson('/api/tickets', $datosPayload);

        // 3. VERIFICACIÓN: Comprueba el 201 Created y que el ticket exista físicamente
        $response->assertStatus(201)
                 ->assertJsonFragment(['title' => 'Ticket desde el Test POST']);

        $this->assertDatabaseHas('tickets', [
            'title' => 'Ticket desde el Test POST'
        ]);
    }

    public function test_el_escudo_bloquea_tickets_con_categoria_inventada()
    {
        $user = User::create(['name' => 'Test User', 'email' => 'test3@test.com', 'password' => '123456']);

        $datosMalos = [
            'title' => 'Ticket Hacker',
            'description' => 'Intento saltarme la validación',
            'category_id' => 9999, // ID QUE NO EXISTE
            'user_id' => $user->id
        ];

        $response = $this->postJson('/api/tickets', $datosMalos);

        // Verifico que devuelve 422 y que el error es por culpa de 'category_id'
        $response->assertStatus(422)
                 ->assertJsonValidationErrors(['category_id']);
    }
}
```

Antes de ejecutar el test debo instalar el driver de SQLite para mi versión de PHP:

sudo apt update

sudo apt install php-sqlite3

```
@ValentinuxX →/workspaces/PoC-Exportation-Project (develop) $ sudo apt install php-sqlite3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  php8.3-sqlite3
The following NEW packages will be installed:
  php-sqlite3 php8.3-sqlite3
0 upgraded, 2 newly installed, 0 to remove and 123 not upgraded.
Need to get 36.0 kB of archives.
After this operation, 159 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 php8.3-sqlite3 amd64 8.3.6-0ubuntu0.24.04.6 [34.2 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 php-sqlite3 all 2:8.3+93ubuntu2 [1844 B]
Fetched 36.0 kB in 0s (107 kB/s)
Selecting previously unselected package php8.3-sqlite3.
(Reading database ... 59710 files and directories currently installed.)
Preparing to unpack .../php8.3-sqlite3_8.3.6-0ubuntu0.24.04.6_amd64.deb ...
Unpacking php8.3-sqlite3 (8.3.6-0ubuntu0.24.04.6) ...
Selecting previously unselected package php-sqlite3.
Preparing to unpack .../php-sqlite3_2%3a8.3+93ubuntu2_all.deb ...
Unpacking php-sqlite3 (2:8.3+93ubuntu2) ...
Setting up php8.3-sqlite3 (8.3.6-0ubuntu0.24.04.6) ...

Creating config file /etc/php/8.3/mods-available/sqlite3.ini with new version

Creating config file /etc/php/8.3/mods-available/pdo_sqlite.ini with new version
Setting up php-sqlite3 (2:8.3+93ubuntu2) ...
Processing triggers for php8.3-cli (8.3.6-0ubuntu0.24.04.6) ...
```

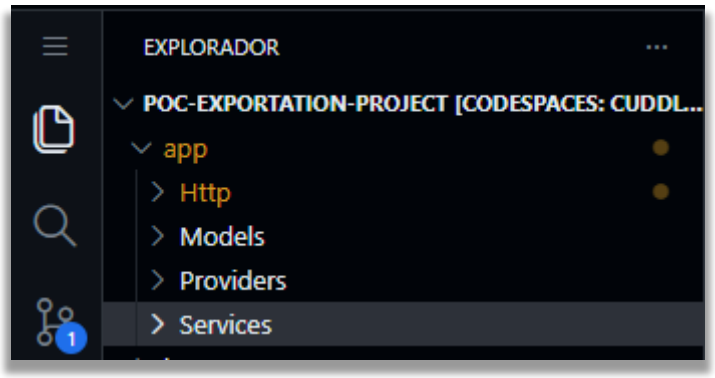
Guardo el archivo y ejecuto los test (con –-filter le indico que solo ejecute este archivo de test y no todos los que trae Laravel por defecto):

php artisan test –-filter TicketApiTest

```
@ValentinuxX →/workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter TicketApiTest

  PASS  Tests\Feature\Api\TicketApiTest
  ✓ puede obtener todos los tickets                                    1.60s
  ✓ puede crear un ticket valido                                       0.01s
  ✓ el escudo bloquea tickets con categoria inventada                  0.01s

  Tests:    3 passed (9 assertions)
  Duration: 1.66s

@ValentinuxX →/workspaces/PoC-Exportation-Project (develop) $
```

En este punto que ya he visto y testeado cómo sería el principio de un CRUD (traer los tickets y crear uno) es el momento en el que decido aislar los servicios en una capa services para seguir el principio de responsabilidad única antes de continuar con el resto del CRUD

Para ello, y ya que Laravel no tiene un comando específico para ello, creo a mano la carpeta /app/Services para albergar los servicios y dejarlos aislados del controlador.



Dentro creo el archivo TicketService.php, que es donde me voy a llevar las consultas a la base de datos:

```php
class TicketService
{
    /**
     * Obtiene todos los tickets con sus relaciones.
     */
    public function getAllTickets()
    {
        return Ticket::with(['user', 'category'])->get();
    }

    /**
     * Crear un nuevo ticket en la base de datos.
     */
    public function createTicket(array $data)
    {
        return Ticket::create($data);
    }
}
```

Ahora, en el controlador solo me voy a ocupar de recibir la petición HTTP, validar la seguridad y devolver la respuesta json. El resto se lo delegaré al servicio mediante inyección de dependencias.

Modifico el controlador:

TicketService.php U | TicketController.php M

app > Http > Controllers > Api > TicketController.php > TicketController

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Services\TicketService; // <-- Importo el servicio

class TicketController extends Controller
{
    protected $ticketService;

    // Inyecto el servicio en el constructor para usarlo en toda la clase
    public function __construct(TicketService $ticketService)
    {
        $this->ticketService = $ticketService;
    }

    public function index()
    {
        // El controlador ya no se encarga de buscar los tickets, solo se los pide al servicio
        $tickets = $this->ticketService->getAllTickets();

        return response()->json($tickets);
    }

    public function store(Request $request)
    {
        // 1. El controlador si asume la responsabilidad de Validar (Seguridad)
        $validated = $request->validate([
            'title' => 'required|string|max:255',
            'description' => 'required|string',
            'category_id' => 'required|exists:categories,id',
            'user_id' => 'required|exists:users,id',
            'status' => 'nullable|string'
        ]);

        // 2. Delega la responsabilidad de Crear al servicio
        $ticket = $this->ticketService->createTicket($validated);

        // 3. Asume la responsabilidad de Responder
        return response()->json($ticket, 201);
    }
```

```php

    // Dejo el resto de funciones em TODO
    public function show(string $id) {}
    public function update(Request $request, string $id) {}
    public function destroy(string $id) {}
}
```

Después de refactorizar ejecuto los test por si he roto algo en el código

Ahora, gracias a los test que cree, al refactorizar es el mejor momento de ejecutarlos por si he roto algo en el código.

```
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter TicketApiTest
  PASS  Tests\Feature\Api\TicketApiTest
  ✓ puede obtener todos los tickets                                    0.54s
  ✓ puede crear un ticket valido                                       0.02s
  ✓ el escudo bloquea tickets con categoria inventada                  0.01s

  Tests:    3 passed (9 assertions)
  Duration: 0.88s
```

Los pasa...

Para terminar el CRUD completo implemento las funcionalidades de traer un solo ticket, actualizar uno existente y, por darlo por concluido, el de eliminar un ticket.

Para ello, siguiendo el patrón de servicios, en el TicketService añado los tres métodos necesarios:

```php
    /**
     * Obtener un ticket específico por su ID.
     */
    public function getTicketById(string $id)
    {
        // findOrFail devuelve un error 404 automático si el ticket no existe
        return Ticket::with(['user', 'category'])->findOrFail($id);
    }

    /**
     * Actualizar un ticket existente.
     */
    public function updateTicket(string $id, array $data)
    {
        $ticket = $this->getTicketById($id);
        $ticket->update($data);
        return $ticket;
    }

    /**
     * Eliminar un ticket de la base de datos.
     */
    public function deleteTicket(string $id)
    {
        $ticket = $this->getTicketById($id);
        $ticket->delete();
        return true;
    }
}
```

Ahora conecto el controlador recibiendo la petición, validar (si hace falta) y delegar al servicio.

```php
    // Dejo el resto de funciones em TODO
    public function show(string $id) {

        // Delego la búsqueda al servicio
        $ticket = $this->ticketService->getTicketById($id);

        return response()->json($ticket, 200);
    }

    public function update(Request $request, string $id) {

        // 1. Valido los datos (uso 'sometimes' para que solo valide lo que el usuario decida enviar)
        $validated = $request->validate([
            'title' => 'sometimes|required|string|max:255',
            'description' => 'sometimes|required|string',
            'category_id' => 'sometimes|required|exists:categories,id',
            'user_id' => 'sometimes|required|exists:users,id',
            'status' => 'sometimes|required|string'
        ]);

        // 2. Delego la actualización al servicio
        $ticket = $this->ticketService->updateTicket($id, $validated);

        return response()->json($ticket, 200);
    }


    public function destroy(string $id) {

        // Delego el borrado al servicio
        $this->ticketService->deleteTicket($id);

        return response()->json(['message' => 'Ticket eliminado correctamente'], 200);
    }
}
```

Creo los test automatizados para las nuevas funcionalidades

TicketService.php U | TicketController.php M | TicketApiTest.php M

tests > Feature > Api > TicketApiTest.php > TicketApiTest > test_puede_borrar_un_ticket

```php
class TicketApiTest extends TestCase
    }

    public function test_puede_obtener_un_solo_ticket()
    {
        $user = User::create(['name' => 'User', 'email' => 'u1@test.com', 'password' => '123']);
        $category = Category::create(['name' => 'Fallo', 'priority' => 'Alta']);

        $ticket = Ticket::create([
            'title' => 'Ticket Único',
            'description' => 'Detalles',
            'category_id' => $category->id,
            'user_id' => $user->id,
            'status' => 'Abierto'
        ]);

        $response = $this->getJson('/api/tickets/' . $ticket->id);

        $response->assertStatus(200)
                 ->assertJsonFragment(['title' => 'Ticket Único']);
    }
```

```php
    public function test_puede_actualizar_un_ticket()
    {
        $user = User::create(['name' => 'User', 'email' => 'u2@test.com', 'password' => '123']);
        $category = Category::create(['name' => 'Fallo', 'priority' => 'Alta']);

        $ticket = Ticket::create([
            'title' => 'Ticket Viejo',
            'description' => 'Vieja',
            'category_id' => $category->id,
            'user_id' => $user->id,
            'status' => 'Pendiente'
        ]);

        $response = $this->putJson('/api/tickets/' . $ticket->id, [
            'title' => 'Ticket Actualizado',
            'status' => 'Resuelto'
        ]);

        $response->assertStatus(200);
        $this->assertDatabaseHas('tickets', [
            'id' => $ticket->id,
            'title' => 'Ticket Actualizado',
            'status' => 'Resuelto'
        ]);
    }
```

```php
    public function test_puede_borrar_un_ticket()
    {
        $user = User::create(['name' => 'User', 'email' => 'u3@test.com', 'password' => '123']);
        $category = Category::create(['name' => 'Fallo', 'priority' => 'Alta']);

        $ticket = Ticket::create([
            'title' => 'Ticket a borrar',
            'description' => 'Adios',
            'category_id' => $category->id,
            'user_id' => $user->id,
            'status' => 'Pendiente'
        ]);

        $response = $this->deleteJson('/api/tickets/' . $ticket->id);

        $response->assertStatus(200);
        $this->assertDatabaseMissing('tickets', [
            'id' => $ticket->id
        ]);
    }
}
```

Corro los test para las funcionalidades que cierran el CRUD

Los pasa:

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter TicketApiTest

   PASS  Tests\Feature\Api\TicketApiTest
   ✓ puede obtener todos los tickets                                    0.50s
   ✓ puede crear un ticket valido                                       0.01s
   ✓ el escudo bloquea tickets con categoria inventada                  0.01s
   ✓ puede obtener un solo ticket                                       0.01s
   ✓ puede actualizar un ticket                                         0.01s
   ✓ puede borrar un ticket                                             0.01s

   Tests:    6 passed (15 assertions)
   Duration: 0.58s

○ @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $
```

Ahora que tengo mi mini API diseñada, implementada y testeada es hora de hacer el merge con mi rama principal.

Guardo los cambios:

git add .

git commit -m "CRUD completo con test automatizados"

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git commit -m "CRUD completo con test automatizados"
[develop 969e0ab] CRUD completo con test automatizados
 3 files changed, 165 insertions(+), 32 deletions(-)
 create mode 100644 app/Services/TicketService.php
```

Cambio a la rama main y hago el merge:

git checkout main

git merge develop

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git merge develop
Updating 897d174..969e0ab
Fast-forward
 app/Http/Controllers/Api/TicketController.php | 77 ++++++
```

git push

git checkout develop

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (main) $ git checkout develop
  Switched to branch 'develop'
  Your branch is ahead of 'origin/develop' by 2 commits.
    (use "git push" to publish your local commits)
○ @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $
```

Una vez construido el CRUD es hora de implementar la funcionalidad de los filtros

Ya que la idea de este proyecto/PoC es la de poder exportar a csv de forma eficiente cuando puede haber cientos de miles de registros, habiendo hecho antes un filtrado por parte del usuario es necesario implementar en el Backend la funcionalidad de los filtros.

Voy a implementar que el usuario pueda filtrar:

- Por rango de fechas
- Por categoría y usuario
- Por prioridad
- Búsqueda de texto

El primer paso es la ruta para esta funcionalidad. La añado al archivo de rutas:

```php
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\TicketController; // importo el controlador

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

// La ruta de exportación
Route::get('/tickets/export', [TicketController::class, 'export']);

// Creo la ruta para la API de tickets (esta ruta crea GET, POST, PUT y DELETE automáticamente)
Route::apiResource('tickets', TicketController::class);
```

Programo el servicio para añadirle el método getTicketForExport para que construya la consulta SQL de forma dinámica según por lo que llegue en el array $filters

```php
     * Obtener los tickets filtrados por múltiples criterios.
     */
    public function getTicketsForExport(array $filters = [])
    {
        // Inicio la consulta base
        $query = Ticket::with(['user', 'category']);

        // 1. Filtro por Estado (Exacto)
        if (!empty($filters['status'])) {
            $query->where('status', $filters['status']);
        }

       // 2. Filtro por Prioridad (Buscando dentro de la tabla categorías)
        if (!empty($filters['priority'])) {
            $query->whereHas('category', function ($q) use ($filters) {
                $q->where('priority', $filters['priority']);
            });
        }

        // 3. Filtro por Categoría (ID exacto)
        if (!empty($filters['category_id'])) {
            $query->where('category_id', $filters['category_id']);
        }

        // 4. Filtro por Usuario (ID exacto)
        if (!empty($filters['user_id'])) {
            $query->where('user_id', $filters['user_id']);
        }

        // 5. Filtro por Rango de Fechas (Creación)
        if (!empty($filters['date_from'])) {
            // "Desde" las 00:00:00 de ese día
            $query->whereDate('created_at', '>=', $filters['date_from']);
        }
```

```php
        if (!empty($filters['date_to'])) {
            // "Hasta" las 23:59:59 de ese día
            $query->whereDate('created_at', '<=', $filters['date_to']);
        }

        // 6. Búsqueda por texto (Título) - "LIKE" es para búsquedas parciales
        if (!empty($filters['search'])) {
            $query->where('title', 'like', '%' . $filters['search'] . '%');
        }

        // Ordenamos por los más nuevos primero
        return $query->orderBy('created_at', 'desc')->get();
    }
}
```

Actualizo el controlador añadiendo el método export añadiendo el BOM (Byte Order Mark) de UTF-8 para que no rompa los caracteres latinos (eñes, tildes, etc...)

```php
    /**
     * Exportar los tickets a un archivo CSV aplicando los filtros recibidos.
     */
    public function export(Request $request)
    {
        // 1. Pedimos los datos al servicio, pasándole los filtros de la URL (Request)
        $tickets = $this->ticketService->getTicketsForExport($request->all());

        // 2. Preparamos las cabeceras HTTP para forzar la descarga del archivo CSV
        $headers = [
            "Content-type"        => "text/csv; charset=UTF-8",
            "Content-Disposition" => "attachment; filename=tickets_export.csv",
            "Pragma"              => "no-cache",
            "Cache-Control"       => "must-revalidate, post-check=0, pre-check=0",
            "Expires"             => "0"
        ];

        // 3. Creamos la función "Stream" que irá escribiendo el archivo línea a línea en la memoria
        $callback = function() use ($tickets) {
            $file = fopen('php://output', 'w');

            // Añado el BOM (Byte Order Mark) de UTF-8.
            // Esto le dice a Microsoft Excel que el archivo tiene acentos y eñes para que no rompa el texto.
            fputs($file, chr(0xEF) . chr(0xBB) . chr(0xBF));

            // Escribo la primera fila (Las cabeceras de las columnas)
            fputcsv($file, [
                'ID',
                'Título',
                'Estado',
                'Usuario',
                'Categoría',
                'Prioridad',
```

```php
                'Fecha de Creación'
            ]);

            // Iteramos sobre la colección y escribimos una línea por cada ticket
            foreach ($tickets as $ticket) {
                fputcsv($file, [
                    $ticket->id,
                    $ticket->title,
                    $ticket->status ?? 'Sin estado',
                    // Uso el operador nullsafe (??) por si acaso algún ticket fue borrado de su usuario/categoría
                    $ticket->user->name ?? 'Sin usuario',
                    $ticket->category->name ?? 'Sin categoría',
                    $ticket->category->priority ?? 'N/A', // Saco la prioridad desde la relación con la categoría
                    $ticket->created_at->format('Y-m-d H:i:s')
                ]);
            }

            fclose($file);
        };

        // 4. Devuelvo la respuesta en streaming hacia el navegador del usuario
        return response()->stream($callback, 200, $headers);
    }
}
```

Empiezo a crear los test automatizados para probar la funcionalidad de filtrado y exportación.

Para no mezclar toda la batería de los nuevos test, más exhaustivos ya que es la base de esta PoC, voy a crear un archivo de test nuevo:

php artisan make:test Api/ExportTicketTest

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan make:test Api/ExportTicketTest

  INFO  Test [tests/Feature/Api/ExportTicketTest.php] created successfully.
```

El plan de acción de los test para la funcionalidad

El plan de acción la voy a dividir en varias fases.

Fase 1: Preparación del laboratorio (setUp)

En lugar de crear un usuario y una categoría copiando y pegando el mismo código en cada uno de los 6 tests que voy a hacer, usaré el método setUp() de PHPUnit. Esto creará un "ecosistema" base (con varios tickets de diferentes fechas, estados y categorías) antes de ejecutar cada prueba.

Fase 2: Test Estructural

Test 1: Verificar que el endpoint devuelve un código 200, que el formato es text/csv, y que incluye el BOM de UTF-8 y las cabeceras correctas de las columnas.

Fase 3: Tests de Filtros Aislados (El bisturí)

Test 2: Filtro por status (Asegurar que si pido "Abierto", no me cuela los "Cerrados").

Test 3: Filtro por texto search (Asegurar que encuentra palabras dentro del título del ticket).

Test 4: Filtro por Rango de Fechas date_from y date_to (Asegurar que ignora los tickets viejos).

Test 5: Filtro Relacional por priority.

Fase 4: Test de Estrés (Filtros combinados)

Test 6: El jefe final. Enviar una petición buscando: "Tickets Abiertos + Prioridad Alta + Creados hoy + Que contengan la palabra 'Error'".

Fase 1: Preparar el laboratorio

Programo el test ExportTicketTest con el setUp() que lo que hace es ejecutarse automáticamente antes de cada test, es decir, que cada vez que el robot va a probar un filtro, primero crea 3 tickets exactos en la RAM, hace la prueba, y luego borra todo. Así tengo un entorno 100% predecible.

ExportTicketTest.php U

tests > Feature > Api > ExportTicketTest.php > ExportTicketTest > setUp

```php
<?php

namespace Tests\Feature\Api;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;
use App\Models\User;
use App\Models\Category;
use App\Models\Ticket;

class ExportTicketTest extends TestCase
{
    use RefreshDatabase;

    // Declaro las variables para poder usarlas en todos los tests
    protected $user;
    protected $catAlta;
    protected $catBaja;
    protected $ticketNuevoAbierto;
    protected $ticketViejoCerrado;
    protected $ticketAyerPendiente;

```

ExportTicketTest.php U

tests > Feature > Api > ExportTicketTest.php > ExportTicketTest > setUp

```php
class ExportTicketTest extends TestCase
    /**
     * FASE 1: PREPARACIÓN DEL LABORATORIO
     * Este método se ejecuta automáticamente antes de CADA test.
     */
    protected function setUp(): void
    {
        parent::setUp();

        // 1. Creo dependencias base
        $this->user = User::create(['name' => 'Admin', 'email' => 'admin@test.com', 'password' => '123456']);
        $this->catAlta = Category::create(['name' => 'Fallo Crítico', 'priority' => 'Alta']);
        $this->catBaja = Category::create(['name' => 'Duda', 'priority' => 'Baja']);

        // 2. Creo 3 tickets muy diferentes para poder jugar con los filtros

        // Ticket A: Nuevo, Abierto, Prioridad Alta, contiene la palabra "Error"
        $this->ticketNuevoAbierto = Ticket::create([
            'title' => 'Error al iniciar sesión',
            'description' => 'No me deja entrar',
            'category_id' => $this->catAlta->id,
            'user_id' => $this->user->id,
            'status' => 'Abierto',
            'created_at' => now(), // Hoy
            'updated_at' => now()
        ]);

        // Ticket B: Viejo, Cerrado, Prioridad Baja, contiene la palabra "Consulta"
        $this->ticketViejoCerrado = Ticket::create([
            'title' => 'Consulta sobre facturación',
            'description' => 'Quiero mi factura',
            'category_id' => $this->catBaja->id,
            'user_id' => $this->user->id,
            'status' => 'Cerrado',
            'created_at' => now()->subMonths(2), // Hace 2 meses
            'updated_at' => now()->subMonths(2)
        ]);
```

```php

        // Ticket C: Medio, Pendiente, Prioridad Alta, contiene la palabra "Pantalla"
        $this->ticketAyerPendiente = Ticket::create([
            'title' => 'Pantalla en blanco',
            'description' => 'Todo se ve blanco',
            'category_id' => $this->catAlta->id,
            'user_id' => $this->user->id,
            'status' => 'Pendiente',
            'created_at' => now()->subDays(1), // Ayer
            'updated_at' => now()->subDays(1)
        ]);
    }

    /**
     * FASE 2: TEST ESTRUCTURAL
     * Compruebo que el archivo está bien construido (BOM, Cabeceras, Columnas)
     */
    public function test_exportacion_devuelve_formato_csv_valido_con_bom()
    {
        // ACCIÓN: Hago la petición sin filtros (debería devolver los 3 tickets)
        $response = $this->get('/api/tickets/export');

        // VERIFICACIÓN:
        $response->assertStatus(200)
                 ->assertHeader('Content-Type', 'text/csv; charset=UTF-8');

        $contenido = $response->streamedContent();

        // 1. Verifico que tiene el BOM de UTF-8 al principio del archivo para Excel
        $this->assertStringStartsWith(chr(0xEF) . chr(0xBB) . chr(0xBF), $contenido);

        // 2. Verifico que las cabeceras de las columnas existen
        $this->assertStringContainsString('ID,Título,Estado,Usuario,Categoría,Prioridad,"Fecha de Creación"', $contenido);
```

```php

        // 3. Verifico que al menos el título de uno de los tickets está en el CSV
        $this->assertStringContainsString('Error al iniciar sesión', $contenido);
    }
}
```

Ejecuto la primera prueba, si la pasa verifico que el laboratorio funciona, las bases de datos en memoria se crean correctamente y el archivo físico del CSV está perfecto para Excel.

Corro el test:

php artisan test –filter ExportTicketTest

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter ExportTicketTest

  PASS  Tests\Feature\Api\ExportTicketTest
  ✓ exportacion devuelve formato csv valido con bom

  Tests:    1 passed (6 assertions)
  Duration: 0.65s
```

Lo pasa.

Creo los test para probar cada uno de los filtros por separado para asegurar de que ninguno interfiere con los demás

```php
    /**
     * TESTS AISLADOS
     */

    public function test_exportacion_filtra_correctamente_por_estado()
    {
        // ACCIÓN: Pedir SOLO los tickets 'Abierto'
        $response = $this->get('/api/tickets/export?status=Abierto');
        $response->assertStatus(200);

        $contenido = $response->streamedContent();

        // VERIFICACIÓN:
        $this->assertStringContainsString('Error al iniciar sesión', $contenido); // Es Abierto (Debe estar)
        $this->assertStringNotContainsString('Consulta sobre facturación', $contenido); // Es Cerrado (NO debe estar)
        $this->assertStringNotContainsString('Pantalla en blanco', $contenido); // Es Pendiente (NO debe estar)
    }
```

```php
    public function test_exportacion_filtra_correctamente_por_busqueda_de_texto()
    {
        // ACCIÓN: Buscar la palabra "Pantalla" en el título
        $response = $this->get('/api/tickets/export?search=Pantalla');
        $response->assertStatus(200);

        $contenido = $response->streamedContent();

        // VERIFICACIÓN:
        $this->assertStringContainsString('Pantalla en blanco', $contenido); // Contiene "Pantalla"
        $this->assertStringNotContainsString('Error al iniciar sesión', $contenido); // NO lo contiene
    }
```

```php
    public function test_exportacion_filtra_correctamente_por_rango_de_fechas()
    {
        // PREPARACIÓN: Definir el rango desde ayer hasta hoy
        $ayer = now()->subDays(1)->format('Y-m-d');
        $hoy = now()->format('Y-m-d');

        // ACCIÓN: Filtrar por fecha
        $response = $this->get("/api/tickets/export?date_from={$ayer}&date_to={$hoy}");
        $response->assertStatus(200);

        $contenido = $response->streamedContent();

        // VERIFICACIÓN:
        $this->assertStringContainsString('Error al iniciar sesión', $contenido); // Es de hoy (Debe estar)
        $this->assertStringContainsString('Pantalla en blanco', $contenido); // Es de ayer (Debe estar)
        $this->assertStringNotContainsString('Consulta sobre facturación', $contenido); // Es de hace 2 meses (NO debe estar)
    }
```

```php
    public function test_exportacion_filtra_correctamente_por_prioridad_relacional()
    {
        // ACCIÓN: Pedir tickets con prioridad 'Baja' (Recuerda que esto busca en la tabla Categorías)
        $response = $this->get('/api/tickets/export?priority=Baja');
        $response->assertStatus(200);

        $contenido = $response->streamedContent();

        // VERIFICACIÓN:
        $this->assertStringContainsString('Consulta sobre facturación', $contenido); // Categoría Baja
        $this->assertStringNotContainsString('Error al iniciar sesión', $contenido); // Categoría Alta
    }
}
```

Corro el test para verificar que cada filtro individual funciona.

```
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter ExportTicketTest

  FAIL  Tests\Feature\Api\ExportTicketTest
  ✓ exportacion devuelve formato csv valido con bom
  ✓ exportacion filtra correctamente por estado
  ✓ exportacion filtra correctamente por busqueda de texto
  ⨯ exportacion filtra correctamente por rango de fechas
  ✓ exportacion filtra correctamente por prioridad relacional
  ─────────────────────────────────────────────────────────────────────────────
  FAILED  Tests\Feature\Api\ExportTicketTest > exportacion filtra correctamente por rango de fechas
  Expected: ID,Título,Estado,Usuario,Categoría,Prioridad,"Fecha de Creación"\n
  1,"Error al iniciar sesión",Abierto,Admin,"Fallo Crítico",Alta,"2026-02-24 13:00:57"\n
  2,"Consulta sobre facturación",Cerrado,Admin,Duda,Baja,"2026-02-24 13:00:57"\n
  ... (2 more lines)

  Not to contain: Consulta sobre facturación

  at tests/Feature/Api/ExportTicketTest.php:143
    139
    140         // VERIFICACIÓN:
    141         $this->assertStringContainsString('Error al iniciar sesión', $contenido); // Es de hoy (Debe estar)
    142         $this->assertStringContainsString('Pantalla en blanco', $contenido); // Es de ayer (Debe estar)
  ➜ 143         $this->assertStringNotContainsString('Consulta sobre facturación', $contenido); // Es de hace 2 meses (NO debe estar)
    144     }
    145
    146     public function test_exportacion_filtra_correctamente_por_prioridad_relacional()
    147     {

  1   tests/Feature/Api/ExportTicketTest.php:143


  Tests:    1 failed, 4 passed (20 assertions)
  Duration: 0.61s
```

Me falla 1 y por lo que veo que me ha sacado en el Excel

2,"Consulta sobre facturación",Cerrado,Admin,Duda,Baja,"2026-02-24 13:00:57”

Por lo que veo que Laravel es muy "sobreprotector" con las fechas. Cuando usas Ticket::create(), el ORM (Eloquent) de Laravel ignora por completo cualquier fecha que le pases y la sobrescribe automáticamente con el segundo exacto en el que se ejecuta el código. Lo hace por seguridad, pero esto en fastidia los test.

Para “engañar” a Laravel y poder escribir tickets en el pasado para los test voy a crearlo de forma normal y luego, usando la clase DB (Que va directo a la base de datos y se salta Eloquent), le cambio la fecha.

Añado el use para poder usar la clase DB:

ExportTicketTest.php U

tests > Feature > Api > ExportTicketTest.php > ...

```php
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;
use App\Models\User;
use App\Models\Category;
use App\Models\Ticket;
use Illuminate\Support\Facades\DB;
```

Y cambio el método setup():

```php
    protected function setUp(): void
    {
        parent::setUp();

        // 1. Creo dependencias base
        $this->user = User::create(['name' => 'Admin', 'email' => 'admin@test.com', 'password' => '123456']);
        $this->catAlta = Category::create(['name' => 'Fallo Crítico', 'priority' => 'Alta']);
        $this->catBaja = Category::create(['name' => 'Duda', 'priority' => 'Baja']);

        // 2. Creo los tickets normales (Laravel les pondrá la fecha de hoy por defecto)

        // Ticket A: Nuevo
        $this->ticketNuevoAbierto = Ticket::create([
            'title' => 'Error al iniciar sesión',
            'description' => 'No me deja entrar',
            'category_id' => $this->catAlta->id,
            'user_id' => $this->user->id,
            'status' => 'Abierto',
        ]);
```

```php
        // Ticket B: Viejo
        $this->ticketViejoCerrado = Ticket::create([
            'title' => 'Consulta sobre facturación',
            'description' => 'Quiero mi factura',
            'category_id' => $this->catBaja->id,
            'user_id' => $this->user->id,
            'status' => 'Cerrado',
        ]);
        // Voy directo a la base de datos para cambiar la fecha y saltarme a Laravel
        DB::table('tickets')->where('id', $this->ticketViejoCerrado->id)->update(['created_at' => now()->subMonths(2)]);

        // Ticket C: Medio
        $this->ticketAyerPendiente = Ticket::create([
            'title' => 'Pantalla en blanco',
            'description' => 'Todo se ve blanco',
            'category_id' => $this->catAlta->id,
            'user_id' => $this->user->id,
            'status' => 'Pendiente',
        ]);
        // Lo muevo al día de ayer
        DB::table('tickets')->where('id', $this->ticketAyerPendiente->id)->update(['created_at' => now()->subDays(1)]);
    }
```

Y pruebo a ejecutar el test a ver si los cambios en la fecha han dado resultado:

php artisan test –-filter ExportTicketTest

```
● @ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter ExportTicketTest

  PASS  Tests\Feature\Api\ExportTicketTest
  ✓ exportacion devuelve formato csv valido con bom
  ✓ exportacion filtra correctamente por estado
  ✓ exportacion filtra correctamente por busqueda de texto
  ✓ exportacion filtra correctamente por rango de fechas
  ✓ exportacion filtra correctamente por prioridad relacional

  Tests:    5 passed (20 assertions)
  Duration: 0.58s
```

Los test pasaron.

Preparo los test para probar los filtros combinados

Programo el test para testear los filtros cuando se usan combinados:

```php
public function test_exportacion_aplica_multiples_filtros_simultaneamente()
{
    // ACCIÓN: Busco tickets que cumplan TODO esto a la vez:
    // - Prioridad: Alta
    // - Estado: Abierto
    // - Creado: Hoy
    // - Texto: "Error"

    $hoy = now()->format('Y-m-d');

    $queryString = http_build_query([
        'priority' => 'Alta',
        'status' => 'Abierto',
        'date_from' => $hoy,
        'search' => 'Error'
    ]);

    $response = $this->get('/api/tickets/export?' . $queryString);
    $response->assertStatus(200);

    $contenido = $response->streamedContent();

    // VERIFICACIÓN:
    // 1. El Ticket A ("Error al iniciar sesión") cumple absolutamente todo. DEBE estar.
    $this->assertStringContainsString('Error al iniciar sesión', $contenido);

    // 2. El Ticket B ("Consulta sobre facturación") falla en prioridad, estado, fecha y texto. NO debe estar.
    $this->assertStringNotContainsString('Consulta sobre facturación', $contenido);

    // 3. El Ticket C ("Pantalla en blanco") es prioridad Alta, pero falla en estado, fecha y texto. NO debe estar.
    $this->assertStringNotContainsString('Pantalla en blanco', $contenido);
}
```

Corro el test:

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter ExportTicketTest
  PASS  Tests\Feature\Api\ExportTicketTest
  ✓ exportacion devuelve formato csv valido con bom
  ✓ exportacion filtra correctamente por estado
  ✓ exportacion filtra correctamente por busqueda de texto
  ✓ exportacion filtra correctamente por rango de fechas
  ✓ exportacion filtra correctamente por prioridad relacional
  ✓ exportacion aplica multiples filtros simultaneamente

  Tests:    6 passed (24 assertions)
  Duration: 0.57s
```

Los 6 test pasaron.

Ahora que ya tengo la API con los test automatizados y la exportación de CSV y que la funcionalidad está terminada hago el commit y el merge antes de poblar la base de datos con las Factories y los Seeders.

Guardo el trabajo:

git add .

git commit -m “API con test automatizados y exportación a CSV”

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git add .
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git commit -m "API con test automatizados y exportación a CSV"
  [develop 564efb3] API con test automatizados y exportación a CSV
   4 files changed, 306 insertions(+)
   create mode 100644 tests/Feature/Api/ExportTicketTest.php
○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

Voy a la rama principal y hago el merge:

git checkout main

git merge develop

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git checkout main
  Switched to branch 'main'
  Your branch is up to date with 'origin/main'.
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git merge develop
  Updating 969e0ab..564efb3
  Fast-forward
   app/Http/Controllers/Api/TicketController.php |  58 ++++++++++++++++++++++++++++++++
   app/Services/TicketService.php                |  50 ++++++++++++++++++++++++++++
   routes/api.php                                |   3 ++
   tests/Feature/Api/ExportTicketTest.php        | 195 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
   4 files changed, 306 insertions(+)
   create mode 100644 tests/Feature/Api/ExportTicketTest.php
○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $
```

Actualizo el repositorio remoto y vuelvo a la rama de desarrollo:

git push

git checkout develop

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git push
Writing objects: 100% (15/15), 5.27 KiB | 5.27 MiB/s, done.
Total 15 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/ValentinuxX/PoC-Exportation-Project
   969e0ab..564efb3  main -> main
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git checkout develop
Switched to branch 'develop'
Your branch is ahead of 'origin/develop' by 3 commits.
  (use "git push" to publish your local commits)
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git push
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ValentinuxX/PoC-Exportation-Project
   eaedefe..564efb3  develop -> develop
○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

Creo Factories para las categorías y para los tickets para construir datos falsos

Ejecuto los comandos para crear los archivos Factory

php artisan make:factory CategoryFactory

php artisan make:factory TicketFactory

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan make:factory CategoryFactory

  INFO  Factory [database/factories/CategoryFactory.php] created successfully.

● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan make:factory TicketFactory

  INFO  Factory [database/factories/TicketFactory.php] created successfully.

○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

Una vez creados los Factory usaré la librería integrada fake() para crear nombres, textos, emails, etc... inventados.

CategoryFactory.php U

database > factories > CategoryFactory.php > ...

```php
<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

/**
 * @extends \Illuminate\Database\Eloquent\Factories\Factory<\App\Models\Category>
 */
class CategoryFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            // Invento una palabra para el nombre de la categoría
            'name' => fake()->unique()->word(),
            // Elijo una prioridad al azar de esta lista
            'priority' => fake()->randomElement(['Alta', 'Media', 'Baja']),
        ];
    }
}
```

Hago lo mismo para el TicketFactory:

TicketFactory.php U

database > factories > TicketFactory.php > ...

```php
/**
 * @extends \Illuminate\Database\Eloquent\Factories\Factory<\App\Models\Ticket>
 */
class TicketFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            // Una frase aleatoria sin punto final para el título
            'title' => fake()->sentence(4, true),
            // Un par de párrafos falsos ("Lorem ipsum...")
            'description' => fake()->paragraph(2),
            // Un estado al azar
            'status' => fake()->randomElement(['Abierto', 'Pendiente', 'Resuelto', 'Cerrado']),

            // Si no le pasamos un ID concreto al crear el ticket, creará uno nuevo automáticamente
            'category_id' => \App\Models\Category::factory(),
            'user_id' => \App\Models\User::factory(),
        ];
    }
}
```

Siembro la base de datos con 50 tickets aleatorios

En el archivo DatabaseSeeder voy a configurarlo para que cree un usuario “admin” (para que pueda iniciar sesión en el futuro), 3 categorías fijas y 50 tickets aleatorios.

DatabaseSeeder.php M

database > seeders > DatabaseSeeder.php > DatabaseSeeder

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use App\Models\Category;
use App\Models\Ticket;
use Illuminate\Database\Seeder;
// use Illuminate\Database\Console\Seeds\WithoutModelEvents;

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // 1. Creo mi usuario administrador de pruebas
        $admin = User::factory()->create([
            'name' => 'Admin PoC',
            'email' => 'admin@test.com',
            // La contraseña por defecto de Laravel en los factories siempre es 'password'
        ]);

        // 2. Creao 3 categorías fijas reales para tener filtros coherentes
        $categorias = [
            Category::create(['name' => 'Fallo Crítico', 'priority' => 'Alta']),
            Category::create(['name' => 'Soporte Técnico', 'priority' => 'Media']),
            Category::create(['name' => 'Duda General', 'priority' => 'Baja']),
        ];

        // 3. Fabrico 50 tickets usando el TicketFactory
        foreach (range(1, 50) as $index) {
            Ticket::factory()->create([
                'user_id' => $admin->id,
                // Elijo una categoría aleatoria del array que cree en el paso 2
                'category_id' => $categorias[array_rand($categorias)]->id,
            ]);
        }
    }
}
```

Ejecuto el comando que borra todas las tablas, las vuelve a crear vacías e inserta mi usuario, las 3 categorías y los 50 tickets.

php artisan migrate:fresh –seed

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan migrate:fresh --seed

  Dropping all tables ..................................................... 57.26ms DONE

  INFO  Preparing database.

  Creating migration table ................................................. 7.75ms DONE

  INFO  Running migrations.

  0001_01_01_000000_create_users_table ..................................... 28.01ms DONE
  0001_01_01_000001_create_cache_table ..................................... 16.15ms DONE
  0001_01_01_000002_create_jobs_table ...................................... 20.77ms DONE
  2026_02_19_095515_create_categories_table ................................. 2.86ms DONE
  2026_02_19_095826_create_tags_table ....................................... 3.26ms DONE
  2026_02_19_110415_create_tickets_table ................................... 17.90ms DONE
  2026_02_19_110620_create_ticket_tag_table ................................ 21.03ms DONE
  2026_02_19_110724_create_attachments_table ............................... 13.11ms DONE
  2026_02_21_071513_create_personal_access_tokens_table .................... 20.23ms DONE


  INFO  Seeding database.


  BadMethodCallException

  Call to undefined method App\Models\Ticket::factory()

  at vendor/laravel/framework/src/Illuminate/Support/Traits/ForwardsCalls.php:67
     63      * @throws \BadMethodCallException
     64      */
     65     protected static function throwBadMethodCallException($method)
     66     {
  ➜  67         throw new BadMethodCallException(sprintf(
     68             'Call to undefined method %s::%s()', static::class, $method
     69         ));
     70     }
     71 }

     +3 vendor frames

  4  database/seeders/DatabaseSeeder.php:34
     Illuminate\Database\Eloquent\Model::__callStatic()
     +35 vendor frames

  40 artisan:16
     Illuminate\Foundation\Application::handleCommand()
```

Me da error ya que olvidé usar el HasFactory en los modelos.

Se lo añado a ambos modelos:

DatabaseSeeder.php M | Category.php M

app > Models > Category.php > ...

```php
<?php

namespace App\Models;


use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    use HasFactory;
    protected $fillable = ['name', 'priority'];

    public function tickets()
    {
        return $this->hasMany(Ticket::class);
    }
}

```

DatabaseSeeder.php M | Category.php M | Ticket.php M

app > Models > Ticket.php > Ticket > attachments

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Ticket extends Model
{
    use HasFactory;

    protected $fillable = [
        'user_id', 'category_id', 'title', 'description',
        'status', 'citizen_name', 'ip_address', 'completed_at'
    ];

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function category()
    {
```

Vuelvo a correr el comando:

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan migrate:fresh --seed

  Dropping all tables ..................................................... 38.53ms DONE

   INFO  Preparing database.

  Creating migration table ................................................. 5.13ms DONE

   INFO  Running migrations.

  0001_01_01_000000_create_users_table ..................................... 23.97ms DONE
  0001_01_01_000001_create_cache_table ..................................... 15.01ms DONE
  0001_01_01_000002_create_jobs_table ...................................... 18.10ms DONE
  2026_02_19_095515_create_categories_table ................................. 2.86ms DONE
  2026_02_19_095826_create_tags_table ....................................... 3.34ms DONE
  2026_02_19_110415_create_tickets_table ................................... 18.94ms DONE
  2026_02_19_110620_create_ticket_tag_table ................................ 15.55ms DONE
  2026_02_19_110724_create_attachments_table ............................... 10.71ms DONE
  2026_02_21_071513_create_personal_access_tokens_table .................... 19.92ms DONE


   INFO  Seeding database.
```

Ya puedo hacer el commit, el merge y el push en git para empezar ya con el frontend en Vue

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git add .
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git commit -m "Añadidos los factories y los seeder para entorno de desarrollo"
[develop b912ac6] Añadidos los factories y los seeder para entorno de desarrollo
 5 files changed, 89 insertions(+), 9 deletions(-)
 create mode 100644 database/factories/CategoryFactory.php
 create mode 100644 database/factories/TicketFactory.php
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git merge develop
Updating 564efb3..b912ac6
Fast-forward
 app/Models/Category.php                |  3 +++
 app/Models/Ticket.php                  |  3 +++
 database/factories/CategoryFactory.php | 26 ++++++++++++++++++++++++
 database/factories/TicketFactory.php   | 32 ++++++++++++++++++++++++++++++++
 database/seeders/DatabaseSeeder.php    | 34 ++++++++++++++++++++++++++---------
 5 files changed, 89 insertions(+), 9 deletions(-)
 create mode 100644 database/factories/CategoryFactory.php
 create mode 100644 database/factories/TicketFactory.php
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git push
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 2 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (12/12), 2.31 KiB | 2.31 MiB/s, done.
Total 12 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To https://github.com/ValentinuxX/PoC-Exportation-Project
   564efb3..b912ac6  main -> main
○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $
```

Y vuelvo a la rama de desarrollo para empezar ya con el Frontend.